



Universidad de Murcia

Facultad de Informática

TRABAJO FIN DE MÁSTER

Análisis de establecimiento de asociaciones de seguridad IPSec usando IKE en entornos SDN

Autor

José Padilla Martínez

Directores

Rafael Marín López

Fernando Pereñíguez García

MURCIA, A 16 DE JUNIO DE 2021

De acuerdo al reglamento que regula los trabajos de fin de máster en la Facultad de Informática de la Universidad de Murcia, declaro que asumo la originalidad del trabajo presentado y que todas las fuentes utilizadas han sido debidamente citadas.

Fdo: José Padilla Martínez.

A handwritten signature in black ink, appearing to read 'José Padilla', with a long horizontal stroke extending to the right.

Murcia, a día 16 de junio de 2021.

Índice

Resumen	7
Extended abstract	9
1. Introducción	13
2. Antecedentes	15
2.1. <i>Software Defined Networking</i> (SDN)	15
2.2. <i>Network Configuration Protocol</i> (NETCONF)	17
2.3. <i>Internet Protocol Security</i> (IPSec)	18
2.4. <i>Internet Key Exchange</i> (IKE)	21
2.5. Contextualización del trabajo	22
3. Objetivos y metodología	25
4. Diseño de la solución	27
4.1. Módulo controlador	27
4.1.1. Diseño de máquina de estados	27
4.1.2. Ambas configuraciones se aplican correctamente	29
4.1.3. Solo una configuración se aplica correctamente	29
4.1.4. Ninguna configuración se aplica correctamente	30
4.2. Modelo YANG	30
4.2.1. Modelo para IKE	31
4.2.2. Modelo para SPD	32
4.2.3. Modelo para PAD	34
4.2.4. Ejemplo de configuración de un fichero XML	35
5. Resultados experimentales	37
5.1. Topología <i>mesh</i>	37
5.2. Topología estrella	41
6. Conclusiones y vías futuras	45
Bibliografía	47

Índice de figuras

1.	Arquitectura general de SDN	16
2.	Flujo entrante y saliente en IPSec	19
3.	Modos de funcionamiento IPSec [16]	20
4.	Negociación inicial de IKE.	21
5.	Máquina de estados que representa el funcionamiento interno del controlador al instanciar políticas de IKE en dos dispositivos	28
6.	Representación de una topología <i>mesh</i> (con 5 nodos)	38
7.	Tiempos medios para establecer una topología <i>mesh</i>	39
8.	Tiempos obtenidos en todas las pruebas realizadas para obtener la topología <i>mesh</i> completa, junto a la progresión obtenida	40
9.	Representación de una topología estrella (con 5 nodos)	41
10.	Tiempos medios para establecer una la topología estrella	42
11.	Tiempos obtenidos en todas las pruebas realizadas para obtener la topología estrella completa, junto a la progresión obtenida	44

Fragmentos de código

1.	Estructura YANG para IKE	31
2.	Estructura YANG para SPD	32
3.	Estructura YANG para PAD	34
4.	Mensaje XML de ejemplo	35

Resumen

La arquitectura de la red actual posee ciertas deficiencias, en especial en temas de rendimiento y seguridad. Estos problemas se van viendo aumentados constantemente por el incesante crecimiento de usuarios y servicios que hacen uso de la red, que desemboca en una pobre calidad de servicio y una saturación de la red. Estos problemas se agravan, por ejemplo, a la hora de realizar actualizaciones, mantenimiento o gestión de seguridad en los diferentes nodos que forman la red. Por ello, el desarrollo de este Trabajo Fin de Máster se va a centrar en intentar mejorar estas carencias, centrándose en aspectos de seguridad, como gestión de material criptográfico para creaciones de canales seguros entre distintos elementos de la red o la creación y distribución de este material.

Partiendo de estas premisas, entra en escena el paradigma SDN (*Software Define Networking*), diseñado para intentar evitar los problemas más relevantes de la arquitectura de red usada actualmente. La idea fundamental de SDN es la abstracción de la capa de administración frente a la capa de datos, centralizando toda la funcionalidad en un único punto de la red. Usando esta configuración, un administrador de red podría configurar todos los nodos desde el mismo punto haciendo uso de un mismo protocolo, sin necesidad de realizar configuraciones individuales o para pequeños grupos. De esta manera, se consigue una arquitectura uniforme y homogénea, mejorando notablemente la versión inicial de la misma. Toda la funcionalidad estará centralizada en una nueva entidad llamada *Controlador*, cuya misión será comunicarse con los diferentes nodos finales de la red para indicarles información relacionada con enrutamiento o políticas de seguridad.

Haciendo uso del modelo SDN en la arquitectura, se pretende conseguir una notable mejora en la gestión de la red en todos en aspectos, como movilidad o enrutamiento. Debido a que este paradigma está todavía en auge, requiere de mucha investigación y desarrollo, especialmente en el ámbito de la seguridad. Por ello, a lo largo de este documento se va a hacer énfasis en la seguridad, debido a que es algo primordial en la red, pero no se ha conseguido todavía un consenso global ni estándares oficiales a la hora de proteger las diferentes comunicaciones. Para el desarrollo de este trabajo, se usa como punto de partida el Trabajo Fin de Grado realizado por el autor de este Trabajo Fin de Máster, en relación a la gestión de asociaciones de seguridad, donde el objetivo actual es mejorar y automatizar el escenario obtenido en el trabajo anterior, optimizando la funcionalidad de los distintos elementos del escenario, en especial la parte del controlador. Se incluye también un análisis en profundidad de los diferentes componentes de la solución, los cuales son el modelo YANG, la incorporación de un controlador y las diferentes pruebas de rendimiento realizadas.

Si bien es cierto que con la propuesta del trabajo comentado anteriormente se

consigue una aplicación de políticas automática en los nodos finales, todavía es necesario desplegar manualmente los componentes del escenario e introducir las diferentes políticas a mano, simulando el comportamiento que tendría un controlador. Como propuesta para solventar esta problemática, se va a analizar en detalle un nuevo módulo controlador que se va a introducir en el escenario, realizando tareas como registrar o aplicar asociaciones de seguridad a los diferentes nodos de la topología. Se analizará el comportamiento general del mismo, centrándose en una máquina de estados usada por el módulo para su correcto funcionamiento, de manera independiente de la implementación. De esta manera, podrá enviar políticas de seguridad directamente a los nodos, quedando automatizado todo el proceso de envío y aplicación de las mismas. Finalmente, cuando tengamos toda la arquitectura desplegada, se podrán obtener diferentes resultados experimentales que nos permitan analizar el rendimiento que ofrece el controlador en diferentes casos de uso. También se analizará el modelo *YANG* utilizado para comprobar la correcta estructura de los mensajes.

A lo largo del documento se van a ir mostrando las distintas tecnologías y trabajos relacionados con esta vía de trabajo e investigación. Se comentará también un diseño teórico de la solución y se concluirá con un análisis de los resultados obtenidos en diferentes experimentos. Por último, es importante remarcar la importancia de seguir investigando en diferentes vías que quedan abiertas tras el desarrollo de este trabajo. Entre otras, podemos destacar la necesidad de continuar el desarrollo y aplicación de esta idea en otros tipos de escenarios así como la implementación de la última versión del modelo *YANG* en el escenario.

Extended abstract

In recent years, there has been a trend in society towards the use of technology and, in particular, the use of the functions offered by the network. This trend is influenced by the advantages of its use, such as wide coverage, efficient mobility, and high data exchange rates. For these reasons, the services or functionalities offered by companies or any other type of entity are increasingly based on making use of the network, resulting in a steady and progressive increase in network requirements. Therefore, this increase in its use also leads to a constant increase in the complexity of network maintenance and management, and there is also a growing need to improve the different techniques and protocols used for this purpose.

Among the different difficulties that can be observed at present, we can see for example the need to unify the different operations to be performed in the network, either in a protocol or standard or in a reduced series of these. Another problem is to manage the entire network from the point of view of a network administrator, where it currently requires the configuration and maintenance for each device or small set of these, differently for each of these. Finally, another obstacle to overcome is the individuality of each of the network components, without taking into account the global behavior or the behavior of nearby nodes. As a result, we have a scenario that is highly complex to manage, not very scalable, and with a static appearance. In short, this scenario represents a constant challenge to be overcome by the network, which must be able to renew and improve at all times, trying to offer the highest possible quality of service for the maximum number of connected devices. To meet these needs, new generations of communication technologies are being developed periodically, which include all kinds of improvements in terms of protocols, speed, and security, among others. However, the advances that have been achieved in the management of network architecture have not generated enough improvement, and this work will focus on trying to provide improvements and solutions to this specific area.

Under these conditions, the SDN (*Software Defined Networking*) paradigm enters the scene, a concept that aims to solve the most important shortcomings of the current network architecture. This paradigm is not something new, since it has been under development for several years, but it is now, with the growing need to improve the network, when the improvements and benefits to be obtained by using this paradigm have become more visible. Its objective is to remove as much complexity as possible from the tasks related to the management of the network architecture, allowing global traffic control and dynamic adjustment of network behavior. To achieve this, it proposes separating the control and data planes, abstracting network management from the infrastructure itself. A new entity is also introduced in the topology, called controller, which will allow total and centralized management of the different devices that make up the entire network. Moreover, since this entity has a global view of the entire network, routing and security decisions can be made in

real-time, for example, diverting traffic to a different route if congestion is observed at a specific point along the planned path, or communicating and instantiating security policies among all the nodes of a new route to be followed by specific traffic.

This paradigm has a structure organized in three different planes. On the one hand, we have the *application plane*, which is the highest plane in the structure and tries to abstract the administrator from the complexity of network management, showing a simple interface that allows configuring and consulting operations, data, and information in general. This plane will communicate with the *control plane* through the *Northbound* interface. On the other hand, we have the control plane, which is located in the center of the topology and has, at least, one controller. This controller will have a global view of the topology and will be able to communicate with the rest of the elements of the architecture. Firstly, it can communicate with the application plane with the previously mentioned interface, secondly, it can communicate with the different controllers through the *Eastbound* and *Westbound* interfaces, and finally, it can communicate with the data plane using the *Southbound* interface. Here appears the third plane called the *data plane*, which has all the end nodes of the network. These nodes will only have forwarding capability and will follow the guidelines set by the controller.

This new approach provides great homogeneity in network management, giving it great flexibility and scalability. So far, progress has been made in a wide variety of fields related to this paradigm, such as quality of service or high availability, but progress in the field of security has lagged. With this in mind, a more in-depth study will be carried out on the security of communications between the different nodes, applying different security associations between the nodes of the network and checking that the communication established is secure (generating traffic among the different elements and observing that the traffic are protected). From this point, two research aspects arise, the first one where the negotiations of security policies would be managed directly and the second one where the negotiation of keys would be delegated to a protocol. In our case, we will focus on the second variant, and the other variant will be investigated by another colleague.

When developing the project, we are going to focus specifically on the Southbound interface, where we will have scenarios composed of a single controller and a variable number of end nodes. To perform the communications in this interface, we make use of an API that provides us with mechanisms to perform a simple communication between the control and data planes. The API works with the NETCONF protocol, which allows us to configure, install, query, or delete end-device configurations. At this point, it is also necessary to make use of a protocol that meets the requirements of our scenario in the pure security domain. For this purpose, and based on previous work, we have continued to use the IPSec protocol suite. Recall that this set of protocols secures communications over IP, authenticating and encrypting

each IP packet exchanged in data traffic. It has two different modes of operation, which are tunnel mode and transport mode, where the former protects the traffic between two intermediate nodes of the communication and the latter protects end-to-end. On the other hand, it also provides a specific protocol for the establishment of security associations, called IKE (in our case we will use version two of the protocol). For the development of our work, we will make use of IKE, delegating to it the initial negotiation of keys when there is no previous negotiation between the different devices. Finally, we will also make use of YANG models, used to check the correct content of the exchanged messages.

The development carried out in this practice does not start from scratch, since it has been used as a starting point in the previous final degree work from the same author of this project, in which a module was designed that was responsible for applying the configurations received in hot, that is, in the final nodes and without having to restart any service or modify any configuration file. Therefore, the starting point of this work is a scenario where the end nodes can apply the configuration automatically when received, but the configuration sent by the controller must be entered manually. However, the tests of the previous work were performed in scenarios with only two nodes. Based on these assumptions, the work will focus on the analysis, understanding, and inclusion of a new module in the controller, and testing in a scenario with more entities. That module is aware of all the nodes in the network, and will be able to connect to each one of them and share the necessary configuration in each case, autonomously. The controller must be aware at all times of the location of the different nodes to be controlled, instantiating the corresponding policies when necessary and without the need to intervene manually for each of the nodes to be configured. An analysis of the YANG model used will also be made. When the scenario is fully deployed, different performance tests will be performed and the results obtained will be analyzed.

Concerning the operation of the policies and their applications, the controller will have to form a message in XML format with the necessary information and send it to the corresponding node. For this purpose, messages created in XML format will be used, which will have to be validated against a YANG model to check that the information entered is consistent. The controller will have to send this information to the two nodes that want to create a secure communication channel between them, and then these nodes will make use of the information received to communicate, finally establishing the desired secure channel. When the process is finished, all traffic exchanged between the two nodes will be protected, and the process will be repeated for all pairs of nodes that want to protect their communications. On the other hand, it is also necessary for the controller to be aware of the existence of the different nodes and for them to be able to communicate with the controller for this purpose. To achieve this, the controller will be waiting for registration messages and the nodes will be sending registration requests, until the controller is aware of their

presence and registers them. When they are registered, a response is sent indicating that they have been registered and the nodes stop sending the registration request. From this point on, the controller has communication with each node and can start instantiating the different security policies.

Throughout this document, we will begin by analyzing the current state of network architecture, along with the motivation for the use of the different technologies and how they work. On the other hand, we will also talk about the methodology used and the different objectives we have in mind in the realization of the work. Subsequently, I will make a theoretical design of the solution we are going to carry out and then the experimental results obtained with our scenario will be shown. We will also review the process from the moment the scenario is started and the controller is aware of the existence of the nodes until the configurations have been correctly applied and secure traffic can be exchanged. The tests will be developed in two scenarios, a first scenario where all the nodes will have security associations with all the others and a second scenario where only one node will have security associations with all the others. To deploy this scenario, virtual containers will be used to represent each instance of the scenario. In the same way, both data networks and control networks will be simulated, all using Docker.

It is expected that the development of this work can help in the understanding and improvement of all areas of security, allowing greater confidence in SDN architectures and faster implementation. Its implementation will allow a generalized improvement for all the entities that make use of the services in the network, such as end-users, companies, or third-party services. Among other things, a more secure, much more scalable, much more efficient, or much simpler to manage architecture will be achieved, compared to the current network architecture. This paradigm is presented as the present and the future of the global network architecture, which necessarily implies that further research and progress must be made in all possible areas. In the development of this work, we have focused on security, mainly due to the scarce research developed in this area, but this does not imply that everything related to security is covered. Different lines of research remain open after the completion of the work, such as testing in other types of scenarios and architectures, the use of other configurations or other modes of deployment, the possibility of editing the configurations already applied, or the use of the latest version of YANG data models.

1. Introducción

El crecimiento de la sociedad se está centrando cada vez más en tecnologías que hacen uso de la red para su funcionamiento, lo que está generando un constante aumento en la demanda de recursos y servicios que hacen uso de la red, necesitando una mejora global en todos los aspectos. Se observa una gran problemática a la hora de administrar todos los elementos que la componen, ya que cada elemento es independiente del resto, complicando la gestión de enrutamiento o de seguridad. Por ello, el desarrollo de este trabajo se centrará en intentar facilitar las tareas de administración relacionadas con la seguridad.

Para subsanar este problema surge el paradigma SDN (*Software Defined Networking*), cuyo objetivo se centra en permitir un mejor control de los dispositivos de la red, desacoplando los planos de datos y de control. De esta forma, se deja a los dispositivos finales únicamente la capacidad de decisión en función de unas reglas, que a su vez aplica un controlador central. Las decisiones que tome el controlador serán mucho más lógicas y precisas que las tomadas de forma independiente por los dispositivos, ya que este tiene una vista global de la red y puede realizar la toma de decisiones en tiempo real. SDN ofrece una oportunidad para definir formas flexibles y escalables de gestionar la seguridad en las redes.

Paralelo al avance en el desarrollo de este tipo de arquitecturas, surgen también diferentes escenarios que requieren una forma centralizada de gestionar la red. Concretamente, se centran más en aspectos relacionados con la seguridad, teniendo como objetivo la protección de todo el tráfico de datos intercambiado entre los distintos dispositivos, donde en particular se presentan diferentes escenarios con la necesidad de gestionar las asociaciones de seguridad IPSec. Un ejemplo podrían ser las redes WAN definidas por software (*SD-WAN*), extensión de SDN que proporciona una abstracción de software para crear redes seguras sobre entornos WAN [1]. Otro ejemplo podrían ser los escenarios relacionados con centros de datos ubicados en la nube, donde la administración dinámica de claves es crítica para la distribución de políticas, establecidas usando IPSec.

El trabajo que realiza manualmente un administrador también se puede delegar en un código que lo realice de forma automática. De esta manera, también se puede automatizar el proceso de registro y comunicación entre los diferentes nodos finales y el controlador. Esta idea ha sido el punto de partida del desarrollo de este trabajo. La investigación no parte de cero, debido a que este trabajo parte del desarrollo realizado en el Trabajo Fin de Grado realizado por el mismo autor de este trabajo, centrado en el desarrollo de un módulo para la gestión y automatización de políticas recibidas, es decir, ejecutado en los nodos finales [2].

En el punto inicial del desarrollo del trabajo inicial, se apreciaban dos posibles

variantes a la hora de gestionar las asociaciones. Una primera opción se centra en la delegación de la negociación de claves en un protocolo ajeno al controlador, y una segunda opción que estaría orientada en la gestión directa de toda la información y la negociación del material criptográfico necesario [3]. El desarrollo del trabajo inicial, y por tanto de esta continuación, centra en el primer caso, y usa el protocolo IKEv2 para la negociación de claves. Por lo tanto, el verdadero punto de partida de este trabajo es un escenario donde disponemos de un controlador y dos nodos de red que implementan IKEv2 para la configuración de IPSec, donde los nodos finales tienen la capacidad de aplicar las configuraciones recibida en caliente (sin necesidad de reiniciar ningún servicio).

Dentro del desarrollo de este trabajo, el objetivo se centrará en conseguir desplegar un escenario lo más autónomo posible, pudiendo analizar de forma completa el rendimiento, realizando una serie de pruebas. Para ello, se va a introducir un módulo en el controlador que sea capaz de comunicarse con los diferentes nodos finales de la red y aplicar las políticas cuando sea necesario (debido a que en el escenario inicial, debemos mandar la configuración manualmente a cada nodo). También se analizará el modelo *YANG* utilizado para comprobar que los mensajes intercambiados en el escenario poseen una estructura correcta.

A lo largo de este trabajo, se van a ir tratando diferentes apartados con información necesaria para la comprensión y la motivación del trabajo desarrollado. En primer lugar, se iniciará con una descripción de las principales tecnologías que componen el proyecto y su estado actual, junto a una contextualización para situar correctamente el proyecto. Después se hará especial hincapié en los principales objetivos del proyecto junto a un repaso de la metodología usada para conseguir esos objetivos. Más adelante, se mostrará un diseño teórico de la solución propuesta para mejorar la gestión de políticas de seguridad, terminando con un análisis de los experimentos realizados y una conclusión final.

2. Antecedentes

Esta sección intenta iniciar al lector en todas las tecnologías que se han usado para el desarrollo del proyecto. Para ello, se comentará para cada una su estado actual, su arquitectura y sus principales componentes. En este caso, hablaremos de SDN en primer lugar, para luego centrarnos en el protocolo usado para llevar las configuraciones a los nodos finales, llamado NETCONF. Después se hablará del protocolo IPSec (solución más extendida en redes IP para ofrecer seguridad a nivel de red), usado para la protección de los datos, junto al protocolo en el que se delega la negociación inicial de claves, llamado IKE. Finalmente, se contextualizará el trabajo, hablando de las posibles vías de desarrollo que surgen al inicio y otros trabajos relacionados desarrollados.

2.1. *Software Defined Networking* (SDN)

El uso del paradigma SDN [4] nos aporta una nueva manera de gestionar la red. Nos permite separar los planos de datos y de control, donde se podrá gestionar toda la red desde un punto centralizado, llamado controlador. El controlador hará uso de su visión global de la red y de las directrices configuradas para poder instanciar correctamente las configuraciones requeridas por cada nodo en cada momento. Con el uso de esta jerarquía centralizada se ofrece una mayor flexibilidad y eficiencia, aportando, por ejemplo, una mayor calidad de servicio.

En relación a la arquitectura, SDN está distribuido en tres planos principales y en otras tres interfaces, permitiendo estas últimas la comunicación entre elementos de diferentes planos. La arquitectura general de SDN se puede ver en la Figura 1, y vamos a empezar hablando de los tres planos que posee, que son el plano de aplicación, el plano de control y el plano de datos:

- **Plano de aplicación:** Esta ubicado en la parte más alta de la topología, donde se encuentran aplicaciones que se comunican con los controladores (plano de control). Estas aplicaciones abstraen al administrador de la topología y su complejidad, permitiéndole gestionarla de una manera más simple y eficiente. Este plano no es objeto de investigación en este proyecto, aunque se considera importante entender el lugar que ocupa en la arquitectura general.
- **Plano de control:** Se encuentra en la parte central de la topología y contiene los controladores de la topología, donde cada uno de ellos puede comunicarse con cualquier otro elemento de la topología, usando las diferentes interfaces disponibles. El controlador podrá recibir instrucciones desde el plano de aplicación y podrá instanciar políticas a los nodos finales de la red, ubicados en el plano de datos.
- **Plano de datos:** Está localizado en la parte baja de la arquitectura y posee los dispositivos finales, como *routers* o *switches*, que únicamente se centran en

dirigir correctamente el tráfico en la red. Cada elemento de este plano seguirá siempre las instrucciones que le indique el controlador, que está situado en la capa superior y posee una vista global de la red. Por lo tanto, los elementos de este plano se limitan únicamente a dirigir el tráfico, sin tomar ninguna decisión propia.

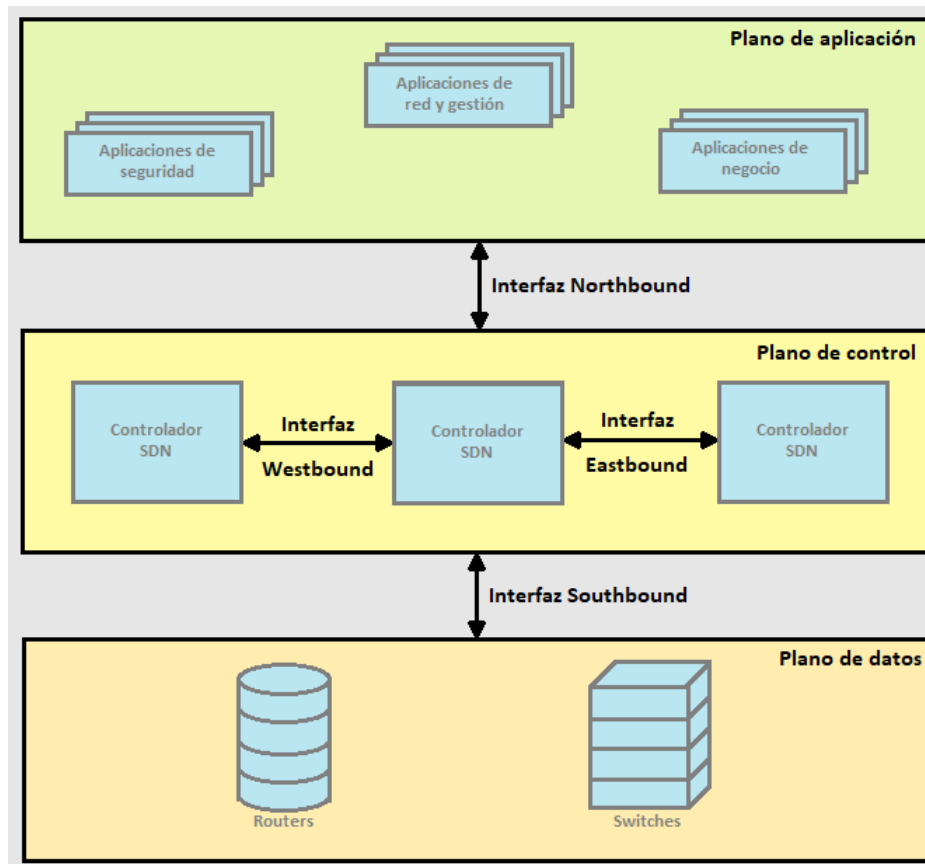


Figura 1: Arquitectura general de SDN

Por otra parte, es importante comprender como se comunican los diferentes elementos que posee la topología con el resto. Para esta comunicación, SDN hace uso de tres interfaces diferenciadas, las cuales son *Northbound*, *Eastbound* & *Westbound* y *Southbound*. Reciben el nombre por los puntos cardinales desde la vista del controlador, y su funcionalidad es:

- **Interfaz *Northbound*:** Su propósito es la comunicación entre el plano de aplicación y el plano de control (entre aplicaciones y controladores). De esta manera, cada vez que el administrador necesite instanciar alguna política, consultar estadísticas o cualquier otra acción, puede comunicarse directamente

con el controlador usando esta interfaz. No se va a usar en el desarrollo de este trabajo, pero es importante saber su función en la arquitectura.

- **Interfaces *Eastbound & Westbound*:** Su objetivo es la comunicación entre diferentes dispositivos controladores, ubicada únicamente en el plano de control. De la misma manera que la interfaz anterior, no entra dentro del ámbito de este trabajo, puesto que solo dispondremos de un controlador y no habrá comunicación con ningún otro controlador.
- **Interfaz *Southbound*:** Pretende simplificar la comunicación entre el plano de control y el plano de datos (entre controladores y elementos finales como *routers* o *switches*). En esta interfaz, el controlador indicará a los diferentes dispositivos como deben distribuir y proteger el tráfico que pase por ellos, y los dispositivos podrán preguntar que hacer con cierto tráfico cuando no lo sepan. Los protocolos más conocidos en esta interfaz son OpenFlow y NETCONF, siendo este último el que usaremos y que explicamos en la siguiente sección.

2.2. *Network Configuration Protocol* (NETCONF)

El protocolo NETCONF [5] apareció para permitir al administrador de la red poder configurar de manera simple y efectiva los distintos elementos que componen la red. Permite la instalación, eliminación y manipulación en general de las diferentes configuraciones, haciendo uso de RPC (*Remote Procedure Call*) [6] para ejecutar el código en una máquina remota. También usa la codificación XML (*Extensible Markup Language*) [7] para el intercambio de mensajes y se validan frente a modelos YANG para verificar el contenido y la sintaxis de la información.

Con respecto a su arquitectura, está distribuida en cuatro capas diferenciadas. Una capa inferior de transporte que nos ofrece rutas de comunicación entre controlador y dispositivo final, y puede usarse sobre protocolos de que cumplan los requisitos (orientados a conexión, soporte de SSH, autenticación, integridad, confidencialidad y protección contra reenvío). Una segunda capa de mensajes que aporta un mecanismo simple e independiente de la capa de transporte para codificar RPC y notificaciones. Una tercera capa de operaciones que invocan métodos de RPC y están codificados en XML (como *get-config* para obtener una configuración concreta o *edit-config* para instanciar una configuración concreta). Por último, tenemos la capa de contenido, que permite la manipulación de los datos de configuración.

Por último, NETCONF define diferentes modelos de datos (*database*) para poder aplicar ciertas operaciones sobre ellos. Estos modelos de datos son un conjunto de operaciones usadas para hacer que el dispositivo que las aplique tenga el estado deseado. Se definen tres modelos de datos, los cuales son *running*, *candidate* y *startup*, tal y como comentamos a continuación:

- ***Database running*:** Contiene en todo momento la configuración actual del

dispositivo. Solo está presente una vez en el dispositivo y está activo siempre que el dispositivo esté funcionando (no estará disponible la información cuando el elemento esté apagado).

- **Database candidate:** Ofrece una configuración que puede ser modificada sin afectar a la configuración actual, pudiendo ser instanciada como configuración actual en cualquier momento (usando la operación *commit*, que provoca que la configuración se replique en *running*). La configuración de este modelo se puede compartir entre varias sesiones, por lo que se debe asumir que otras sesiones pueden modificar la configuración candidata al mismo tiempo.
- **Database startup:** Posee la configuración que se debe cargar cuando arranca el dispositivo, pasando esta a ser también la configuración actual *running* en ese momento. Se dispone también de una operación (*copy-config*) que nos permite copiar en cualquier momento la configuración actual a este modelo.

2.3. Internet Protocol Security (IPSec)

IPSec [8] aparece como un conjunto de protocolos que proporciona seguridad en las comunicaciones en la capa IP, delegando normalmente la negociación inicial de claves en el protocolo IKE. IPSec usa asociaciones de seguridad para formar las diferentes reglas de protección, donde se pueden indicar entre otras cosas algoritmos de cifrado, modos de cifrado y parámetros de red. Aparte de IKE (protocolo que veremos en más detalle en la siguiente sección), se definen también otros dos protocolos encargados de proporcionar seguridad a nivel de paquete, que son AH y ESP.

AH [9] fue diseñado para establecer autenticación e integridad en cada paquete, colocando una firma digital al paquete, evitando así la interferencia de entidades ajenas a la comunicación. Para ello, se calcula un HMAC (*Hash Message Authentication Code*) usando el contenido del propio paquete junto a una clave, usando algoritmos como MD5 (*Message-Digest Algorithm 5*) [10] y SHA1 (*Secure Hash Algorithm 1*) [11]. Puede también, opcionalmente, proteger frente a ataques de repetición. Actualmente, este protocolo ya no se usa.

Por su parte, ESP [12] ofrece autenticación e integridad como AH, pero añade también confidencialidad. Para ello, cifra el contenido del paquete haciendo uso de algoritmos de cifrado como 3DES (*Triple Data Encryption Standard*) [13] y AES (*Advanced Encryption Standard*) [14]. De esta manera, mientras que AH evita la posibilidad de manipular de un paquete, ESP asegura que la información paquete no pueda ser accesible y esté cifrada.

IPSec también define bases de datos para asegurarse un correcto procesamiento del tráfico IP, consultando y actualizando estas, para tener disponible tanto la información de autenticación como de cifrado. Las entidades son SAD, SPD y PAD [15] y se describen a continuación:

- SPD (*Security Policy Database*): Especifica el tráfico que debe ser protegido mediante IPSec y el tráfico que debe dejar pasar, siendo consultada para todo el tráfico entrante y saliente. Se usará la primera política que coincida con los datos del paquete, procesándolo en base a las reglas que contenga. Si no se encuentra ninguna regla para el paquete, será descartado automáticamente.
- SAD (*Security Association Database*): Contiene información relacionada con el procesamiento de IPSec, de tal manera que se comprobará en la entrada correspondiente que protección debe aplicarse al archivo. Por lo tanto, se accederá a esta base de datos en el caso de que SPD indique que el paquete debe ser protegido con IPSec
- PAD (*Peer Authorization Database*): Proporciona un enlace entre el protocolo IKE e IPSec. En este punto se negocian los parámetros necesarios para la seguridad en la comunicación con el destino, con fines de autenticación.

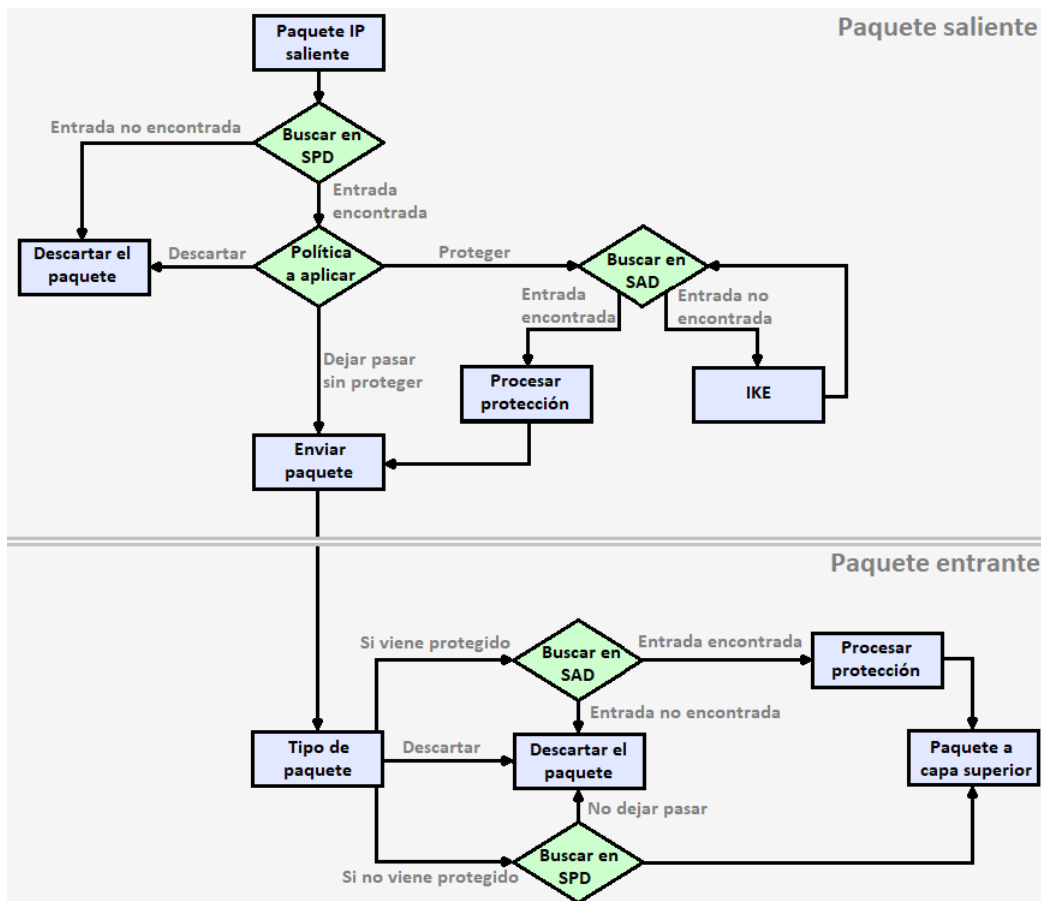


Figura 2: Flujo entrante y saliente en IPSec

Con respecto al funcionamiento del tráfico y de las bases de datos, podemos observar en la Figura 2 el flujo que sigue cada paquete entrante o saliente, haciendo uso de IPSec. Cuando un paquete va a ser enviado, se comprueba si hay alguna entrada en la SPD. Si no se encuentra una entrada correspondiente a ese paquete, se descartará directamente, y si se encuentra una entrada nos indicará tres posibles acciones: descartar, dejar pasar o protegerlo usando IPSec.

Por lo tanto, si la acción es descartar será descartado directamente y si la acción es dejar pasar se enviará al destino. En caso de indicar que se debe proteger con IPSec, se consultará directamente a SAD en busca de una entrada para ese paquete. En caso afirmativo, se procesan las reglas en el paquete y se manda al destino. En caso negativo, se comunica con IKE para gestionar una nueva asociación entre el origen y el destino, y esta regla se guarda en la SAD. Esto implica que, cuando vuelve a consultar a SAD, esta ya se dispone de una regla concreta. Por lo tanto, se aplica la protección indicada en la entrada y se manda el paquete al destino.

En el caso de recibir el paquete, si no viene protegido se consultará directamente en la SPD, la cual descartará o dejará pasar el paquete en función de la entrada. Si viene protegido, se consultará a la SAD, y en caso de no encontrar ninguna entrada descarta el paquete, procesándolo y mandándolo a la capa superior en caso de encontrar una entrada.

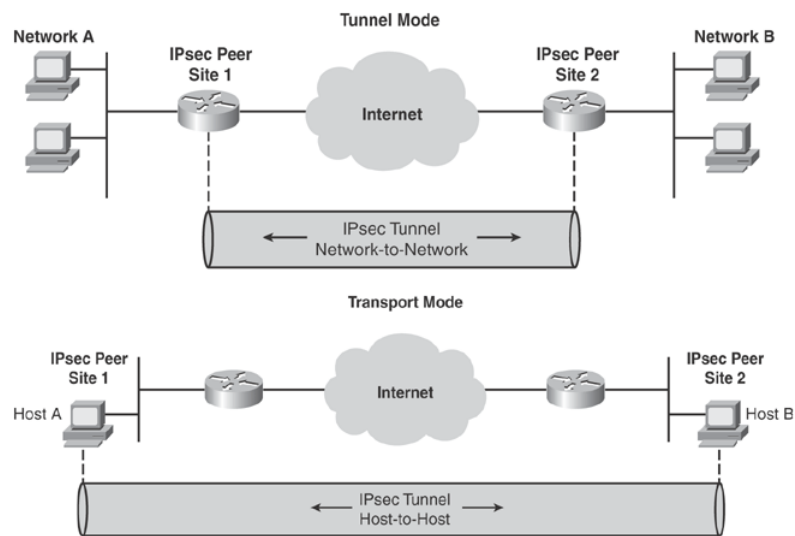


Figura 3: Modos de funcionamiento IPSec [16]

Por último, IPSec posee dos modos de funcionamiento, que son modo túnel y modo transporte (ambos funcionales tanto con AH como con ESP). Estos modos se denominan túnel y transporte y se pueden ver de forma gráfica en la Figura 3. Las

principales características de ambos son:

Por parte del modo túnel, es el modo predeterminado de funcionamiento de IPSec. Protege la totalidad del paquete, cifrando el paquete original y añadiendo una cabecera con la dirección IP del otro extremo del túnel. Este modo está pensado para enviarse entre entidades no finales, por ejemplo, entre dos *routers* que den acceso a la red a dos organizaciones distintas. Por parte del modo transporte, proporciona protección únicamente de la carga útil del paquete. De esta manera, la carga útil protegida se ubica entre las cabeceras y los campos finales de IPSec, que van si proteger. Este modo está pensado para usarse en comunicaciones directas, es decir, entre dos nodos finales de la red.

2.4. *Internet Key Exchange (IKE)*

IKE [17] es un componente de IPSec que tiene como objetivo realizar una negociación de claves entre dos entidades, con el fin de establecer un canal de comunicación seguro entre ellos. Trabaja con intercambio seguro de claves *Diffie-Hellman* [18] para obtener el secreto compartido de cada sesión, usando normalmente sistemas de clave pública. Trabaja sobre el nivel de aplicación usando UDP, y una vez se han acordado las claves iniciales, estas se pasan a la pila IPSec, que se está ejecutando en el propio *kernel*. IKE también se usa para la renovación de las claves negociadas previamente, una vez estas expiren, volviendo a realizar la negociación.

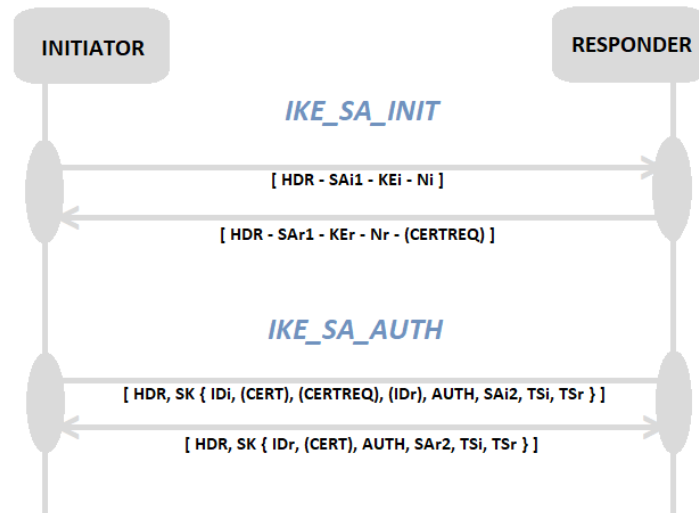


Figura 4: Negociación inicial de IKE.

Para comprender su funcionamiento, vamos a ver los mensajes intercambiados en la negociación inicial y su contenido, que pueden observarse en la Figura 4. En

primer lugar tenemos el intercambio *IKE_SA_INIT*, encargado de negociar los algoritmos criptográficos y realizar la negociación *Diffie-Hellman*. En el envío de estos mensajes se intercambia información como los algoritmos criptográficos aceptados (*SAi1* y *SAr1*) o los valores necesarios para la negociación DH (*Kei*, *NI*, *KEr* y *Nr*). En este punto, ambos poseen la información suficiente para derivar claves que les permitan comunicarse de forma segura, por lo que todos los mensajes intercambiados estarán protegidos (a excepción de las cabeceras).

Acto seguido sucede el intercambio de mensajes *IKE_SA_AUTH*, que tiene como objetivo autenticar los mensajes, intercambiar certificados y generar la primera asociación de seguridad. Dentro de los mensajes, se puede observar tanto el certificado como la petición a la otra entidad del mismo (*CERT* y *CERTREQ*), se intercambian también los parámetros para establecer la asociación de seguridad (*SAi2* y *SAr2*) o los selectores de tráfico (*TSi* y *TSr*).

2.5. Contextualización del trabajo

La organización *Internet Engineering Task Force* (IETF) está liderando uno de los esfuerzos más relevantes actualmente en el ámbito de la gestión de seguridad en redes SDN. Concretamente, el documento [19] propone un esquema de gestión de asociaciones de seguridad IPSec gobernado por una entidad central (controlador). Tal y como se recoge en este documento, IPSec se integra perfectamente en la arquitectura SDN, ya que la operación del protocolo IPSec (plano de datos) se puede desacoplar del protocolo utilizado para la gestión de claves (plano de control), como IKEv2.

La solución propuesta en esta referencia hace uso de NETCONF para la configuración de los dispositivos de red ubicados en el plano de datos y, consecuentemente, define un modelo YANG (que se analizará en secciones posteriores de este documento) que estructura el contenido de la información transportada por los mensajes intercambiados entre controlador y dispositivo. Sin embargo, una de las aportaciones más relevantes de este trabajo reside en los modelos que se proponen para establecer asociaciones de seguridad IPsec, de los cuales se realiza un estudio básico de rendimiento en [20].

Uno de los modelos propuestos delega la negociación inicial de claves en el protocolo IKE, concretamente se hace uso de IKEv2. El controlador se encargará de gestionar y aplicar entradas de las bases de datos SPD y PAD, junto a la información que usará IKE para la negociación de claves (como por ejemplo una clave compartida previamente). De esta manera, IKE se encarga de establecer las asociaciones de seguridad IPSec entre los dispositivos correspondientes. En el desarrollo del trabajo hemos seguido las directrices de este primer caso.

Alternativamente, se propone otro modelo donde la gestión se realiza sin disponer

de ningún protocolo que realice la negociación inicial de las claves ni las asociaciones de seguridad. En este caso, será el controlador el encargado de gestionar las bases de datos SPD y SAD (se gestiona la SAD puesto que el controlador configura directamente las asociaciones de seguridad y no delega en ningún protocolo), eliminando la gestión de PAD puesto que no se hace uso de IKE. Este segundo caso será continuado en otro trabajo de máster, desarrollado de forma paralela a este.

Tomando como referencia este documento de estandarización, el desarrollo de este Trabajo Fin de Máster continua la línea de investigación iniciada por trabajos de trabajos previos relacionados con la gestión de asociaciones de seguridad dentro del paradigma SDN, en concreto siguiendo el modelo que hace uso de IKE para la negociación inicial de claves. En este sentido, es importante destacar el trabajo previo realizado por Ignacio Martínez Alpiste [21] y, muy especialmente, el Trabajo Fin de Grado titulado *Módulo para la gestión de asociaciones de seguridad IPSec usando SDN e IKE en dispositivos de red* [2] realizado por el autor de este Trabajo Fin de Máster. Concretamente, el trabajo que se presenta aquí es una continuación de las contribuciones desarrolladas en este último.

El trabajo previo realizado por el autor de este documento en su TFG se centraba principalmente en la idea de diseñar e implementar un módulo que permitiera la instalación de políticas de seguridad en el dispositivo de red final. De esta manera, se consigue que un nodo pueda aplicar la configuración de forma automática en el mismo momento que la recibe y la procesa, consiguiendo esto sin necesidad de reiniciar ningún servicio ni cambiar ningún fichero de configuración. Este trabajo soluciona el problema en el lado del plano de datos, ya que consigue aplicar las configuraciones instanciadas por el controlador automáticamente, pero el controlador sigue teniendo que instanciar las configuraciones de forma manual. El despliegue y las pruebas de concepto realizadas se basaban en un escenario llamado *host-to-host*, donde se tiene un controlador y dos nodos a configurar. Tanto el trabajo previo como este no hacen uso de las interfaces *Northbound*, *Eastbound* y *Westbound*, centrándose directamente en la interfaz *Shouthbound* que proporciona la comunicación directa entre plano de datos y de control.

Partiendo de esta base, este trabajo pretende, en primer lugar, abarcar un abanico más amplio de escenarios donde realizar pruebas, permitiendo la virtualización de un escenario con un número de nodos finales variables (en comparación a los dos nodos fijos que teníamos en el trabajo anterior). Se pretende también la inclusión de un módulo en el controlador que permita la instanciación de las políticas en cada uno de los nodos cuando sea necesario, eliminando la necesidad de indicar al controlador de forma manual cuando y cómo debe aplicar la configuración a cada uno de los nodos. Se desplegarán también dos escenarios diferenciados para intentar obtener la mayor cantidad de resultados útiles, teniendo un primer escenario donde se evaluará el rendimiento de realizar una topología *mesh* (se refiere a la creación

de asociaciones de seguridad de todos los nodos con el resto, es decir, cada pareja posible de nodos) y un segundo escenario que evaluará el rendimiento al realizar una topología estrella (se refiere a la creación de asociaciones de seguridad de un solo nodo con el resto). Se obtendrán métricas de parámetros relacionados con el tiempo necesario para poder desplegar una topología global securizada en función del número de nodos del escenario, representando la información en distintas gráficas y obteniendo conclusiones de estas.

3. Objetivos y metodología

Este apartado define los diferentes objetivos perseguidos en el trabajo, junto a la metodología usada para conseguir estos objetivos. Para ello, se van a plantear los objetivos del trabajo partiendo del desarrollo del trabajo realizado en el Trabajo Fin de Grado previo.

El primer objetivo es la extensión del trabajo realizado anteriormente, donde partiendo del escenario final presentado por este, se pretende adaptar este a las necesidades que tendría el uso de SDN en escenarios más realistas. Concretamente, busca que el escenario pueda albergar una cantidad indefinida de nodos, y por lo tanto, se acerque más a un escenario realista que el escenario que usábamos en el trabajo anterior.

Otro objetivo marcado es la comprensión del funcionamiento que posee el lado del controlador. En base al trabajo previo, se avanzó bastante en la parte de los nodos finales pero se dejó de lado la parte del controlador. Por ello, se va a analizar un módulo dado por los tutores de este trabajo y se va a incluir en el nodo controlador, que, sea capaz de comunicarse con los distintos nodos finales que tendrá la red y tenerlos localizados, permitiendo instanciar políticas de seguridad cuando sean requeridas, de forma totalmente autónoma. De esta manera, tendremos tanto los nodos como el controlador automatizado, y por lo tanto, todo el escenario que abarca el trabajo.

Por otra parte, el intercambio de información que se va a producir entre los diferentes elementos de la topología necesita también de una estructura bien definida y un control de la misma. Esta información, recordemos que contendrá datos relacionados con el conjunto de protocolos IPSec y el protocolo IKE y se define una estructura válida de esa información usando el modelo YANG. El estudio de este modelo es otro de los objetivos marcados en el desarrollo del trabajo.

Por último, se desarrollarán diferentes pruebas experimentales, que nos permitirán obtener diferentes resultados y que nos revelarán el rendimiento final del sistema de automatización desplegado. Entre otras, se estudiarán métricas relacionadas con el tiempo necesario para tener securizada la comunicación entre todos los nodos de una arquitectura o el tiempo de cada negociación individual. Una vez obtenidos los resultados, se estudiarán y se sacarán conclusiones en relación a estos, reflejando la eficiencia de los escenarios desplegados. Todas estas pruebas se van a realizar mediante escenarios virtualizados, donde se simularán los diferentes elementos físicos que compondrían el escenario.

Continuando con la metodología utilizada para el desarrollo del trabajo y la obtención de los diferentes objetivos, vamos a repasar brevemente los diferentes

puntos seguidos, todo ello en un orden cronológico:

- Se estudiarán las necesidades que tiene el uso del paradigma SDN, de cara a poder dar soluciones útiles en este trabajo que pueda llegar aplicarse en entornos reales. En base a esto, se necesita también comprender las principales tecnologías usadas para su correcto funcionamiento. Partiendo de estas necesidades, se ha modificado el escenario para que se parezca lo máximo posible a uno real.
- Después, aparece la inclusión de un módulo automatizado en el controlador, dotando al escenario final de una mayor velocidad y flexibilidad. También en este punto se creará un método que permite a los diferentes nodos comunicarse con el controlador, consiguiendo que este sea consciente de la existencia de los diferentes nodos y pueda aplicarle las configuraciones necesarias en el momento indicado, de forma autónoma.
- Otro punto a tratar es el estudio y análisis del modelo YANG utilizado en nuestro escenario, debido a que necesitamos de un modelo que nos permita validar la información intercambiada entre el controlador y los nodos, asegurando que la información enviada es correcta. Por otra parte, hacer hincapié también en el análisis de la última versión estandarizada, aunque la versión usada en este trabajo sea anterior.
- Los experimentos se van a realizar sobre un escenario que posee un controlador y un número cualquiera de nodos finales. Dentro de esta topología, vamos a usar dos variantes, que son la realización de una topología *mesh* (es decir, cada nodo asociado con todos los demás) y la realización de una topología estrella (es decir, un sólo nodo se asocia con todos los demás). Se realizarán diferentes experimentos en ambos casos y se obtendrán diferentes métricas, relacionadas con el tiempo necesario para aplicar las configuraciones de forma global o de forma individualizada, entre otras métricas.
- Se analizarán los resultados obtenidos y se obtendrán conclusiones relacionadas con el rendimiento en ambos escenarios, y con distinto número de nodos.

4. Diseño de la solución

Una vez hemos desarrollado los diferentes objetivos principales del trabajo, en este apartado se van a desarrollar dos temas fundamentales para el funcionamiento de nuestro escenario, con la idea de poder dejar cubiertos los objetivos comentados previamente en la sección 3.

En primer lugar vamos a exponer el funcionamiento que debe tener el controlador para permitir el funcionamiento deseado, automatizando el proceso de comunicación e instanciación con los nodos finales. Seguidamente, se desarrollan y se definen los modelos usados para estructurar la información intercambiada entre las diferentes entidades en cada uno de los mensajes.

4.1. Módulo controlador

En primer lugar, se va a desarrollar la funcionalidad necesaria para el correcto desarrollo de la actividad del controlador, donde tiene que ser capaz en todo momento de poder conocer a los diferentes nodos que hay disponibles en la topología y comunicarse con ellos para poder configurarlos. Se va a comentar el funcionamiento interno que tendría el controlador cuando se necesita aplicar políticas de IKE entre un par de nodos, usando como base una máquina de estados que describa ese funcionamiento.

4.1.1. Diseño de máquina de estados

El siguiente paso es ver en detalle el funcionamiento interno que tendría el controlador. Para ello, nos vamos a centrar en la instanciación de políticas de IKE, observando el flujo de estado que presenta mediante la máquina de estados representada en la Figura 5. La máquina de estados tiene representadas las transiciones de la forma evento/acción, donde en la parte superior se indica el evento que ha sucedido y en la parte inferior se indica la acción realizada, que ha sido desencadenada por el evento concreto.

Antes de comentar el funcionamiento de la máquina de estados, es importante saber que acciones desempeña cada función asociada a las transiciones. Destacar que en todo momento recibir una notificación de error o no recibir una notificación en un tiempo determinado representan un error en la configuración de la misma manera. En primer lugar tenemos las acciones *actualizar_error* y *actualizar_OK*, y su objetivo principal es poder transicionar al estado necesitado en cada caso. Realmente no tienen ninguna acción asociada, sino que simplemente sirven para ir al estado correspondiente en función de la primera notificación recibida. Será cuando se reciba la segunda notificación cuando se desencadenen eventos.

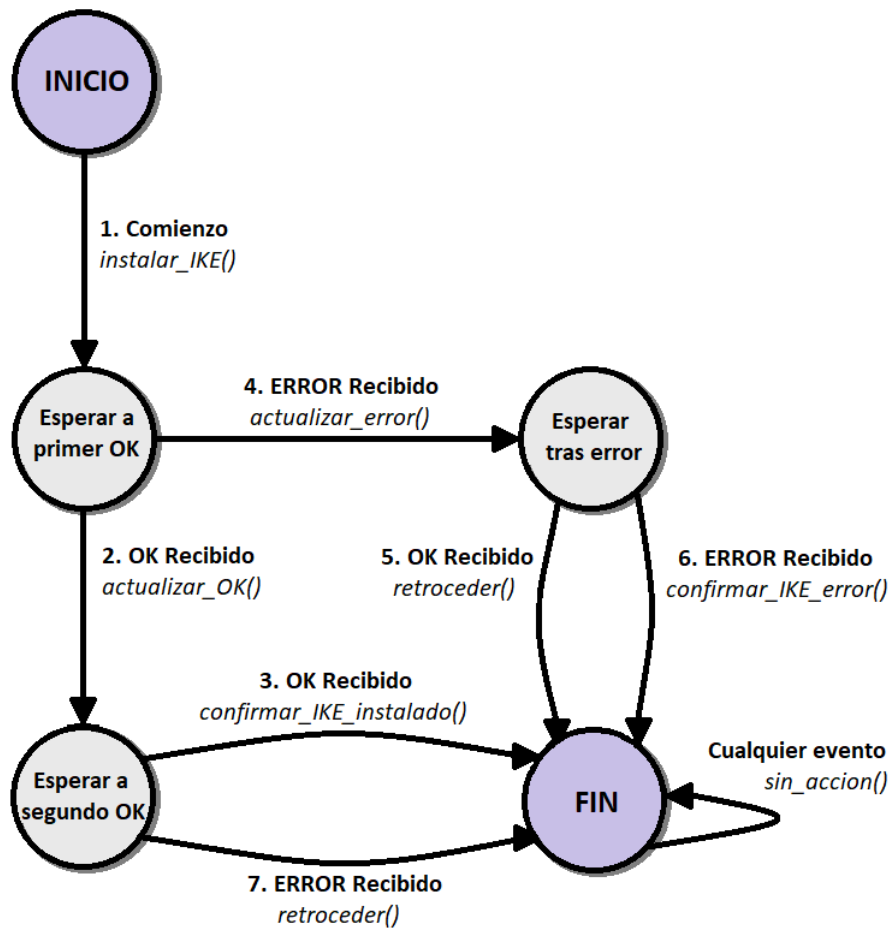


Figura 5: Máquina de estados que representa el funcionamiento interno del controlador al instanciar políticas de IKE en dos dispositivos

Por otra parte, tenemos la acción *retroceder*, desencadenada únicamente cuando se ha recibido una notificación indicando error y otra indicando que todo ha ido bien. El objetivo que tiene es volver al estado previo a la máquina de estados, es decir, se elimina la configuración en el dispositivo que se ha instanciado correctamente y se asume que el dispositivo que ha notificado el error no se recupera, estando de vuelta en el estado inicial. Por otra parte, tenemos el evento *confirmar_IKE_error* que se desencadena cuando ambas notificaciones han indicado error, donde se para el proceso y no se llegan instancian las políticas en ningún dispositivo. Tenemos también la acción *confirmar_IKE_instalado* que se activa cuando se ha completado correctamente la instanciación y permite continuar el flujo de la ejecución del controlador. Finalmente también está representada la acción *sin_accion*, cuya transición es siempre al mismo volver al estado final, incluida por completitud en la representación de la máquina de estados.

Pasando al funcionamiento de la máquina de estados, parte del momento en que el controlador tiene que instanciar las políticas de IKE en un par cualquiera de nodos finales. Cuando esto sucede, la máquina de estados comienza e instala las políticas, donde ambas configuraciones se envían en paralelo, aplicándose la primera transición (*instalar_IKE*) y pasando al siguiente estado, en el cual se esperará una confirmación de correcta instanciación de políticas o una notificación de que ha sucedido un error en el proceso, por parte de cualquiera de los dos nodos. A continuación, tenemos diferentes caminos que puede tomar la máquina de estados, en función de las notificaciones recibidas. Las opciones que tenemos son que ambas configuraciones se hayan aplicado correctamente, ambas hayan fallado o que una se aplique bien y la otra no lo haga. A continuación, se va a ver en detalle cada uno de los posibles casos.

4.1.2. Ambas configuraciones se aplican correctamente

En primer lugar, vamos a comenzar analizando el caso en el que ambas configuraciones se han aplicado correctamente y hemos recibido ambas notificaciones también de forma correcta.

Cuando se recibe una confirmación de que la instanciación de políticas se ha establecido de forma correcta (de cualquiera de los dos nodos, es indiferente cual sea), se desata el evento correspondiente *actualizar_OK* y se pasa al estado que espera una segunda notificación (que puede ser de error o de confirmación pero en este caso será de confirmación). Pasado un breve período de tiempo se recibirá una notificación por parte del otro dispositivo (ya que en este caso asumimos que llegará correctamente y dentro del período de tiempo de espera), indicando que la instanciación ha sido correcta y desatando el último evento (*confirmar_IKE_instalado*).

Finalmente, la máquina de estados llega a su estado final, habiendo instanciado correctamente las correspondientes configuraciones de IKE. En este caso, el recorrido ha pasado por las transiciones: $1 \rightarrow 2 \rightarrow 3$. En caso de no recibir la confirmación en un período de tiempo establecido, se considera un error y se transiciona de igual manera que si se recibe una notificación de error, opción cubierta en las siguientes casos.

4.1.3. Solo una configuración se aplica correctamente

En este caso, se va a analizar el caso donde solo se aplica correctamente la configuración en uno de los dos nodos. Si se recibe primero la notificación de que las políticas se han instanciado bien, se desencadenará la acción *actualizar_OK* y se pasará al estado de espera de la segunda confirmación. Si ahora se recibe una notificación indicando que ha ocurrido un error durante la configuración del otro nodo (o se ha agotado el tiempo de espera asociado), se desencadena la acción *retroceder* y se pasa al estado final, dando por terminada la máquina de estados y volviendo al

estado inicial. Las transiciones seguidas han sido $1 \rightarrow 2 \rightarrow 7$.

El caso contrario sucederá cuando se reciba en primer lugar la notificación de que ha sucedido un error. En ese caso, se transiciona al estado de espera tras recibir un error y se desencadena el evento *actualizar_error*. Acto seguido se recibirá otra notificación indicando que ha ido bien la instanciación de la configuración en el otro nodo y se llegará al estado final desencadenando también el evento *retroceder*, donde se dará por terminada la máquina de estados y se volverá al estado inicial. El flujo de transiciones seguido en este caso ha sido $1 \rightarrow 4 \rightarrow 5$.

4.1.4. Ninguna configuración se aplica correctamente

Por último, queda analizar el caso en el que ambas instanciaciones fallan. Para ello, el controlador será notificado de un error en la instanciación en cualquiera de los dos nodos. Tras recibirla, la máquina de estados avanza al estado de esperar tras un error y se desata el evento *actualizar_error*.

En este punto, se recibirá una segunda notificación (o no se recibirá pasado el tiempo establecido, donde ambas opciones tienen el mismo efecto) correspondiente al otro dispositivo, indicando también que algo ha salido mal al instanciar la configuración. Se desencadena el evento *confirmar_IKE_error* y se termina la máquina de estados, quedando ambas configuraciones sin aplicar y en el mismo punto previo a la máquina de estados. Las transiciones que se han desencadenado en este caso han sido $1 \rightarrow 4 \rightarrow 6$.

4.2. Modelo YANG

Se necesita de un mecanismo que permita estructurar y organizar de una manera homogénea la información intercambiada. Para ello, se hace uso de los modelos YANG, encargado de definir la sintaxis y el contenido de la información intercambiada, permitiendo también validar los mensajes. La configuración contenida en los mensajes intercambiados posee información relacionada con el protocolo IKE y las bases de datos SPD y PAD.

A lo largo de este apartado, se va a analizar la composición del último modelo estandarizado (recordar que en este trabajo no se usa el último modelo y queda como posible vía futura su inclusión), haciendo especial hincapié en los aspectos más relevantes en cada caso. Se va a representar la información en forma de árbol, donde el símbolo *?* indicará que el elemento no es obligatorio, el símbolo *** indicará que puede aparecer más de una vez (normalmente se asocia a una lista de la forma [nombre lista]) y por último, se indica a la derecha de cada elemento el tipo de datos.

4.2.1. Modelo para IKE

En primer lugar, vamos a analizar la composición de atributos que nos ofrece el modelo para la información intercambiada relacionada con IKE. La información vendrá representada en una lista de conexiones llamada *ike-conn-entries*, que contendrá al menos una conexión de IKE. Los diferentes campos son:

```
rw ikev2 {case1}?
+--rw ike-connection
+--rw ike-conn-entries* [conn-name]
  +--rw conn-name          string
  +--rw autostartup         type-autostartup
  ...
  +--rw version?           enumeration
  +--rw ike-reauth-lifetime uint32
  +--rw ike-sa-lifetime    uint32
  +--rw IPSec-sa-lifetime  uint32
  ...
  +--rw local
  | +--rw (my-identifier-type)?
  | | +--:(ipv4)
  | | | +--rw ipv4?          inet:ipv4-address
  | | +--:(ipv6)
  | | | +--rw ipv6?          inet:ipv6-address
  | | ...
  | | +--rw my-identifier    string
  +--rw remote
  | +--rw (my-identifier-type)?
  | | +--:(ipv4)
  | | | +--rw ipv4?          inet:ipv4-address
  | | +--:(ipv6)
  | | | +--rw ipv6?          inet:ipv6-address
  | | ...
  | | +--rw my-identifier    string
  +--rw pfs_group*          uint32
```

Código 1: Estructura YANG para IKE

- *conn-name*: Representa el nombre que tiene cada entrada de conexión de IKE concreta.
- *autostartup*: Atributo que indica si la la aplicación de las políticas se realiza en caliente en el momento que se reciba la configuración o no.
- *version*: Indica la versión concreta de IKE utilizada.
- *ike-reauth-lifetime*: Define el tiempo que ha de pasar para volver a realizar la autenticación de IKE.
- *ike-sa-lifetime*: Representa el tiempo de vida de una asociación de seguridad IKE.
- *IPSec-sa-lifetime*: Es el tiempo de vida asociado a las asociaciones de seguridad de IPSec.

- *local*: Define una serie de atributos para identificar a la entidad local de la asociación de seguridad. Los atributos son, entre otros:
 - *ipv4*: Dirección IPv4 asociada a la entidad local.
 - *ipv6*: Dirección IPv6 asociada a la entidad local en caso de no usar IPv4.
 - *my-identifier*: Cadena de texto identificativa de la entidad local.
- *remote*: De manera análoga a *local*, define una serie de atributos para identificar a la entidad remota de la asociación de seguridad. Los atributos son:
 - *ipv4*: Dirección IPv4 asociada a la entidad remota.
 - *ipv6*: Dirección IPv6 asociada a la entidad remota cuando no usa IPv4.
 - *my-identifier*: Cadena de texto identificativa de la entidad remota.
- *dh-group*: Información relacionada con el grupo *Diffie-Hellman* usado.

4.2.2. Modelo para SPD

Ahora es el turno de analizar los diferentes atributos definidos por el modelo para representar la información relacionada con la SPD. La información de la SPD puede venir en una o varias entradas llamadas *spd-entry*, donde cada una de estas contendrá la información necesaria que debe aplicarse luego en el nodo final. Los atributos principales son:

```
rw spd
+--rw spd-entry* [rule-number]
+--rw rule-number          uint64
+--rw priority?            uint32
...
+--rw condition
| +--rw traffic-selector-list* [ts-number]
| | +--rw ts-number          uint32
| | +--rw direction?         IPSec-traffic-direction
| | +--rw local-addresses* [start end]
| | | +--rw start            inet:ip-prefix
| | | +--rw end              inet:ip-prefix
| | +--rw remote-addresses* [start end]
| | | +--rw start            inet:ip-prefix
| | | +--rw end              inet:ip-prefix
| | +--rw next-layer-protocol* IPSec-next-layer-proto
| | +--rw local-ports* [start end]
| | | +--rw start            inet:port-number
| | | +--rw end              inet:port-number
| | +--rw remote-ports* [start end]
| | | +--rw start            inet:port-number
| | | +--rw end              inet:port-number
| | +--rw selector-priority? uint32
+--rw processing-info
| +--rw action              IPSec-spd-operation
| +--rw IPSec-sa-cfg
...
| +--rw security-protocol?  IPSec-protocol
```



```

|      +--rw mode?                IPSec-mode
|      +--rw ah-algorithms
|      |      +--rw ah-algorithm*  integrity-algorithm-t
|      +--rw esp-algorithms
|      |      +--rw authentication* integrity-algorithm-t
|      |      +--rw encryption*    encryption-algorithm-t
|      +--rw tunnel
|      |      +--rw local?          inet:ip-address
|      |      +--rw remote?        inet:ip-address
|      ...

```

Código 2: Estructura YANG para SPD

- *rule-number*: Un identificador único para cada entrada SPD.
- *priority*: Es un campo opcional que nos permite indicar la prioridad de una entrada concreta en relación al resto de entradas.
- *traffic-selector-list*: Lista que contiene uno o más selectores de tráfico, usados para filtrar el tráfico, donde se descartará todo el tráfico que no cumpla con las directrices indicadas. Los atributos que posee son:
 - *ts-number*: Un identificador único para cada selector de tráfico.
 - *direction*: Es un campo opcional que permite limitar el tráfico en una dirección concreta.
 - *local-addresses*: Se trata de una lista que posee las diferentes rangos de direcciones origen aceptados en el tráfico. Posee atributos de inicio y fin para poder indicar el rango de manera sencilla.
 - *remote-addresses*: De manera similar a la anterior, es una lista que posee las diferentes rangos de direcciones aceptados en el tráfico, aunque en este caso se trata de destino del tráfico y no de origen. También posee atributos de inicio y fin para poder indicar el rango de manera sencilla.
 - *next-layer-protocol*: Atributo que indica el protocolo de transporte usado, generalmente UDP o TCP.
 - *local-ports*: Una lista con rangos de puertos origen aceptados para el tráfico (análogo a *local-addresses*). Posee dos atributos para indicar el inicio y el fin del rango.
 - *remote-ports*: También es una lista con rangos de puertos aceptados con el tráfico, pero esta vez son puertos destino (análogo a *remote-addresses*). Igualmente, posee también dos atributos para indicar inicio y fin de rango.
- *processing-info*: Define un conjunto de atributos que indican como procesar cada paquete, indicando que hacer con el paquete y como hacerlo. Los principales atributos son:
 - *action*: Indica que acción se va a aplicar sobre cada paquete. Las opciones son descartarlo, dejarlo pasar sin proteger (normalmente un nodo intermedio en una comunicación) o protegerlo.

- *IPSec-sa-cfg*: Contiene un conjunto de atributos que nos indica la manera de proteger el paquete, en el caso de indicar el campo *action* que debemos proteger el paquete, donde los atributos más importantes son:
 - *mode*: Atributo opcional para indicar si IPSec funcionará en modo transporte o modo túnel. En caso de no indicar nada funciona en modo túnel por defecto.
 - *security-protocol*: Atributo opcional que nos indica el protocolo de IPSec usado, es decir AH (integridad) o ESP (integridad y confidencialidad).
 - *ah-algorithms*: En caso de usar el protocolo AH, posee un atributo en forma de lista donde se puede indicar el algoritmo o los algoritmos de integridad usados.
 - *esp-algorithms*: En caso de usar el protocolo ESP, contiene dos atributos en forma de lista. Un primer atributo donde se puede indicar el algoritmo o los algoritmos de integridad usados y también un segundo atributo para el algoritmo o los algoritmos de cifrado (confidencialidad) usados.

4.2.3. Modelo para PAD

Por último, vamos a repasar los principales atributos que representan la información relacionada con SAD. Esta información viene representada en una o más entradas de la tabla *pad-entries*, teniendo cada entrada toda la información necesaria de manera independiente. El contenido más relevante de cada entrada de esta tabla es:

```
rw pad {case1}?
+--rw pad-entries* [pad-entry-id]
  +--rw pad-entry-id          uint64
  +--rw (identity)?
    | +--:(ipv4-address)
    | | +--rw ipv4-address?    inet:ipv4-address
    | +--:(ipv6-address)
    | | +--rw ipv6-address?    inet:ipv6-address
    ...
  +--rw pad-auth-protocol?    auth-protocol-type
  +--rw auth-method
    +--rw auth-m?             auth-method-type
    +--rw pre-shared
    | +--rw secret?           string
    ...
```

Código 3: Estructura YANG para PAD

- *pad-entry-id*: Un identificador asociado a cada entrada de PAD.
- *ipv4*: Dirección IPv4 correspondiente a la entidad remota.

- *ipv6*: Dirección IPv6 correspondiente a la entidad remota en caso de no usar direcciones IPv4.
- *pad-auth-protocol*: Protocolo usado para la autenticación (normalmente IKEv2).
- *auth-method*: Define todos los atributos necesarios para representar toda la información relacionada con la autenticación. Entre otros atributos:
 - *auth-m*: Se indica el método de autenticación utilizado, como por ejemplo clave pre-compartida (*pre-shared*).
 - *pre-shared*: Aporta información relacionada con el método concreto de clave pre-compartida. Por ejemplo, se indica el valor de la clave en el campo *secret*.

4.2.4. Ejemplo de configuración de un fichero XML

Por último, se va a mostrar un ejemplo de un XML válido, que podría ser enviado por el controlador a un nodo cualquiera de la red para configurarlo. Este mensaje seguirá la estructura YANG que se ha comentado en los apartados previos.

```
<IPSec nc:operation="merge">
  <spd>
    <spd-entry>
      <rule-number>1</rule-number>
      <names>
        <name>out/192.168.123.100/192.168.123.200</name>
      </names>
      <condition>
        <traffic-selector-list>
          <ts-number>7</ts-number>
          <direction>OUTBOUND</direction>
          <local-addresses>
            <start>192.168.123.100/32</start>
            <end>192.168.123.100/32</end>
          </local-addresses>
          <remote-addresses>
            <start>192.168.123.200/32</start>
            <end>192.168.123.200/32</end>
          </remote-addresses>
          <next-layer-protocol>TCP</next-layer-protocol>
          <local-ports>
            <start>0</start>
            <end>0</end>
          </local-ports>
          <remote-ports>
            <start>0</start>
            <end>0</end>
          </remote-ports>
        </traffic-selector-list>
      </condition>
      <processing-info>
        <action>PROTECT</action>
        <IPSec-sa-cfg>
          <security-protocol>esp</security-protocol>
          <mode>TRANSPORT</mode>
        </IPSec-sa-cfg>
      </processing-info>
    </spd-entry>
  </spd>
</IPSec>
```

```

    </IPSec-sa-cfg>
  </processing-info>
</spd-entry>
</spd>
<pad>
  <pad-entries>
    <pad-entry-id>7</pad-entry-id>
    <ipv4-address>192.168.123.200</ipv4-address>
    <pad-auth-protocol>IKEv2</pad-auth-protocol>
    <auth-method>
      <auth-m>pre-shared</auth-m>
      <pre-shared>
        <secret>secreto_compartido</secret>
      </pre-shared>
    </auth-method>
  </pad-entries>
</pad>
</IPSec>
<ikev2>
  <ike-connection>
    <ike-conn-entries>
      <conn-name>gateway1</conn-name>
      <autostartup>ALWAYSON</autostartup>
      <version>ikev2</version>
      <ike-reauth-lifetime>60</ike-reauth-lifetime>
      <ike-sa-lifetime>30</ike-sa-lifetime>
      <IPSec-sa-lifetime>10</IPSec-sa-lifetime>
      <dh_group>2048</dh_group>
      <local>
        <ipv4>192.168.123.100</ipv4>
        <my-identifier>Ignacio</my-identifier>
      </local>
      <remote>
        <ipv4>192.168.123.200</ipv4>
        <my-identifier>Ruben</my-identifier>
      </remote>
      <pfs_group>0</pfs_group>
      <!--<phase2-lifetime>100</phase2-lifetime>
      <phase2-authalg>hmac-md5-96</phase2-authalg>
      <phase2-encalg>3des</phase2-encalg>-->
    </ike-conn-entries>
  </ike-connection>
</ikev2>
</ietf-IPSec>

```

Código 4: Mensaje XML de ejemplo

A lo largo de este mensaje, se puede ver como se intercambian los parámetros necesarios para establecer una asociación de seguridad con otro nodo, lo que implica que el otro nodo debe recibir también un mensaje análogo para poder establecer la asociación de seguridad. Se puede observar en primer lugar la información relacionada con SPD, seguida de la información perteneciente a PAD y terminando con la información correspondiente a IKE.

5. Resultados experimentales

Durante el desarrollo de esta sección, se va a mostrar el despliegue de los dos casos de estudio que hemos planteado, junto a los diferentes experimentos que hemos realizado en cada uno de ellos. Ambos escenarios se basan en una topología donde se dispone de un solo controlador y un número variable de nodos, y todas las pruebas se han realizado en entornos virtualizados haciendo uso de *Docker*.

Los escenarios que se reproducen son una topología *mesh* y una topología en forma de estrella, sobre los que indagaremos a continuación. El uso de estos escenarios se justifica en la idea de replicar escenarios reales, donde por ejemplo el escenario tipo *mesh* se usa normalmente en *data centers*, y los escenarios tipo estrella son típicos en entornos de SD-WAN. En estos escenarios se realizarán diferentes pruebas experimentales con una cantidad incremental de nodos (en este caso desde 2 hasta 20 nodos), y se analizarán los tiempos necesarios para establecer estos canales seguros, medidos desde el punto de vista del controlador. Es importante tomar estos tiempos desde el controlador, debido a que esta entidad es la que inicia las negociaciones, y la toma de tiempos se puede realizar de una forma más precisa.

También es importante destacar la máquina sobre la que se han realizado las pruebas, ya que podrían variar los resultados obtenidos en función de la capacidad de cómputo de esta. La máquina usada posee 8 CPUs (4 núcleos y 2 hilos de procesamiento por núcleo), junto a una frecuencia de procesador máxima de 2.6 GHz.

5.1. Topología *mesh*

La primera topología usada es la topología *mesh*, cuyo objetivo se centra en conseguir que todos los nodos que componen el escenario puedan comunicarse de forma segura entre ellos. Para ello, cuando el controlador es consciente de todos los nodos que conforman el escenario, se debe comunicar uno a uno e ir instanciando en ellos la configuración necesaria. De esta manera, cada nodo podrá comunicarse con el resto de nodos y establecer una asociación de seguridad, haciendo uso de la información recibida. Cuando todos los nodos hayan establecido las asociaciones de seguridad correspondientes con el resto de nodos, se puede decir que tenemos una topología *mesh* desplegada. Se puede ver un ejemplo representativo (en este caso con 5 nodos) en la Figura 6.

Se han realizado diferentes experimentos, con el objetivo de medir en el tiempo que se tarda en establecer la topología completa. Es decir, las pruebas se centran en saber el tiempo que se necesita para disponer de una topología totalmente securizada, entendiéndose esto como la diferencia de tiempo entre que el controlador empieza el proceso de comunicarse con cada nodo para instanciar las configuraciones de seguridad hasta que ha terminado de instanciarlas en el último de estos.

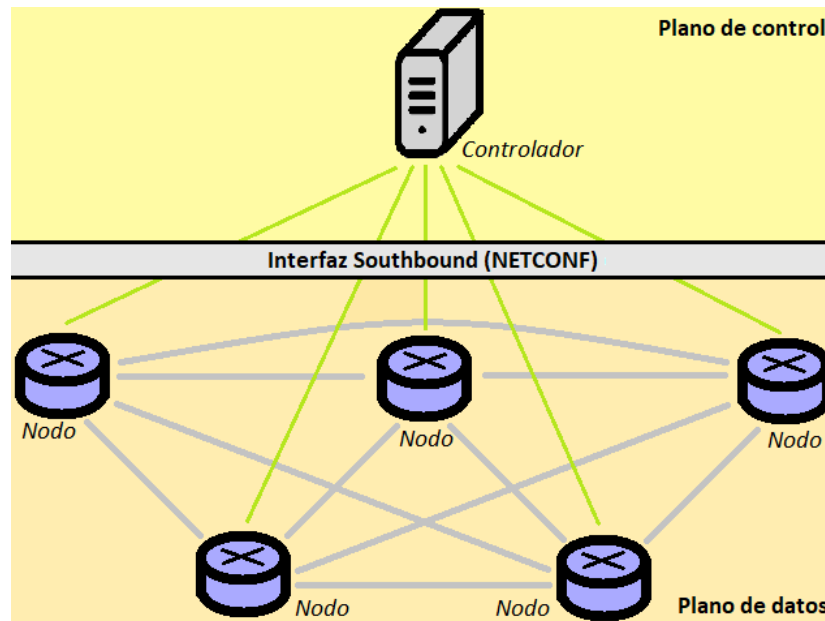


Figura 6: Representación de una topología *mesh* (con 5 nodos)

Se han realizado una serie de experimentos para obtener los tiempos relacionados con la creación de la topología completa. En concreto, se han realizado pruebas para establecer una *mesh* de 2 hasta 20 nodos, con la idea de comprobar como varía este tiempo en función del aumento del número de nodos en el escenario. Para que estos resultados sean más representativos, se han realizado 30 experimentos para cada número de nodos concreto, pudiendo obtener un valor más preciso del tiempo empleado en cada caso. Los resultados obtenidos tras realizar todas estas pruebas se muestran en la Figura 7.

Como primera impresión, se observa que el crecimiento de los tiempos en función del aumento de los nodos no sigue una distribución lineal, sino que se aproxima más a una distribución cuadrática. Esta distribución de tiempos, se debe entre otros motivos a que el número de conexiones y de políticas a aplicar tampoco aumenta de una manera lineal. Por ello, a la hora de analizar los resultados obtenidos, es importante saber también como aumenta el trabajo que se debe realizar en función del número de nodos, es decir, como aumenta el número de políticas a aplicar (y por tanto el número de conexiones necesarias con los distintos nodos de la red).

Tal y como está implementado el controlador, una vez tiene a todos los nodos preparados para recibir las configuraciones, va procediendo con ellos de forma secuencial (realmente se lanzan en hilos paralelos pero esta parte se realiza en exclusión mutua, por lo que en la práctica se realiza de forma secuencial), de tal manera que

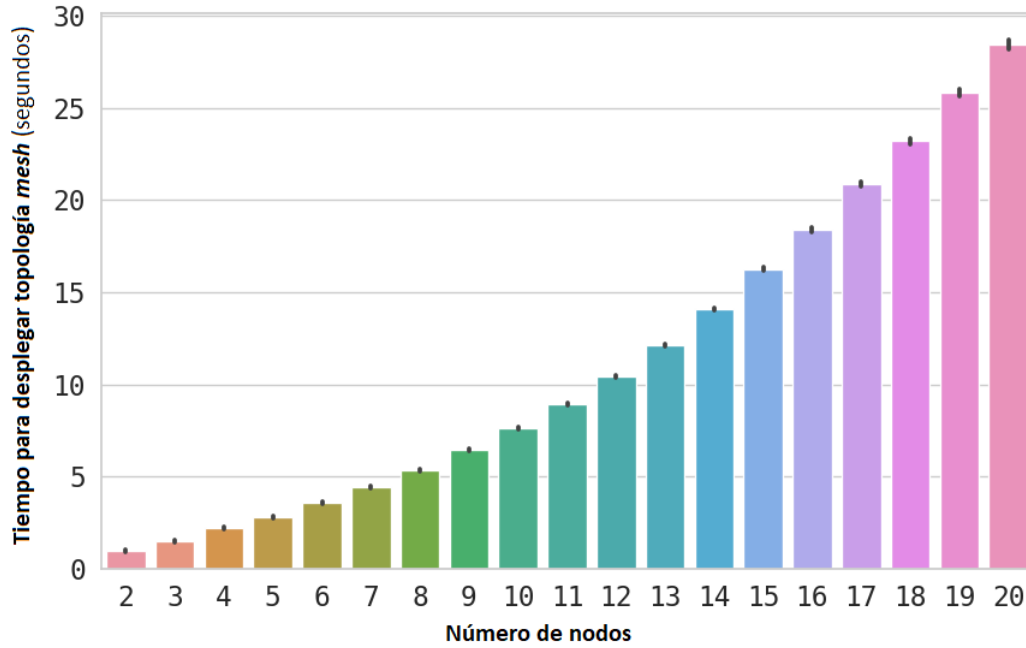


Figura 7: Tiempos medios para establecer una topología *mesh*

cuando tiene que configurar cada dispositivo se comprueba el número de nodos que se han configurado antes, y el nodo actual se configura con cada uno de los nodos previos solamente. Por ejemplo, cuando es el turno del primer nodo, como no hay ningún nodo previo con el que configurarse, cuando entra el segundo nodo, ya hay disponible un nodo y se configuran las políticas en ambos (teniendo que aplicar configuración para una sola comunicación bidireccional). Cuando entra el tercero, hay dos nodos previos y se aplican las configuraciones con ambos (teniendo en este caso que establecer dos comunicaciones bidireccionales nuevas, siendo un total de tres si tenemos en cuenta la que ya se ha realizado), y así sucesivamente.

De esta manera, se puede deducir que cada vez que entra un nodo nuevo a configurarse con el resto, el número de asociaciones de seguridad que se crean (canales de comunicación bidireccionales) son la suma de todos los que había antes más los que crea el nuevo nodo con el resto (es decir, $n-1$ canales siendo n el número actual de nodos configurados). De la misma manera que obtenemos el sumatorio de $n-1$ como el número de asociaciones, podemos deducir que para n nodos, las asociaciones de seguridad a crear se obtienen de la forma $n*(n-1)/2$ (se puede demostrar por inducción matemática). Por lo tanto, siendo n el número de nodos de la topología *mesh* que se quiere desplegar, se puede decir que las comunicaciones bidireccionales que se deben crear son:

$$\sum_{i=1}^n (n-1) = \frac{n(n-1)}{2}$$

De esta manera, al igual que el crecimiento de la gráfica sigue una distribución cuadrática, el aumento de las asociaciones de seguridad en función del número de nodos y el despliegue de la topología *mesh* también sigue un orden $O(n^2)$.

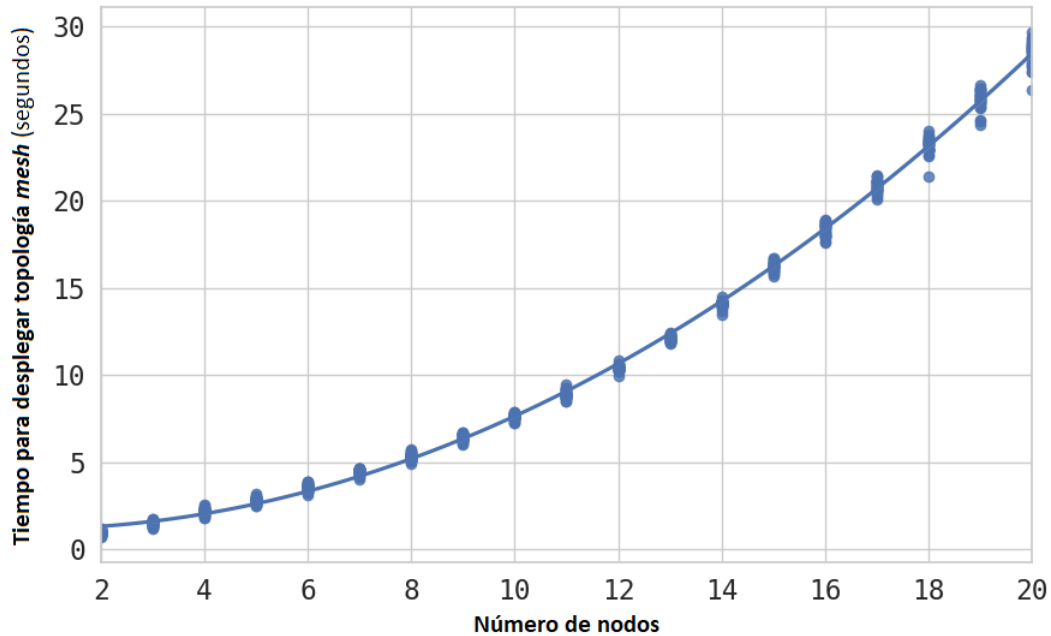


Figura 8: Tiempos obtenidos en todas las pruebas realizadas para obtener la topología *mesh* completa, junto a la progresión obtenida

Si bien es cierto que las configuraciones a aplicar en función del número de nodos aumentan de una manera no lineal, es fácil ver que los tiempos no aumentan de una manera tan abultada como lo hacen las configuración a aplicar. En los tiempos influyen más aspectos aparte del aumento de las configuraciones a aplicar. Entre otras, se destaca la realización de todas las aplicaciones de políticas de forma paralela en distintos hilos de procesamiento (como se comentaba antes, hay algunas partes realizadas en exclusión mutua, pero la mayoría del procesamiento se hace de forma paralela real). La progresión que se ha obtenido, junto a todas las pruebas que se han realizado para cada número de nodos (representadas cada una como un punto en la gráfica), se pueden observar en la Figura 8. Comprobando la información presentada en esta gráfica, se puede ver como sigue la tendencia cuadrática comentada previamente.

Por otra parte, se puede ver también en todos los resultados obtenidos como tienen un nivel de dispersión muy bajo, obteniendo valores similares en cada una de las pruebas que se han realizado, siendo en este caso 30 por cada número de nodos. Los tiempos obtenidos en el experimento son bastante estables, debido principalmente

a que el intervalo de confianza es pequeño. Para el caso de 20 nodos, el tiempo de formar una *mesh* es de aproximadamente unos 27 segundos. Por lo tanto, para obtener el tiempo necesario (de forma aproximada) para formar una *mesh* de n nodos, se puede obtener un valor medio por asociación de seguridad, ya que disponemos de $n*(n-1)/2$ asociaciones de seguridad en el escenario.

5.2. Topología estrella

La segunda topología que se ha usado para hacer pruebas experimentales es la topología estrella, cuyo objetivo se basa en conseguir que todos los nodos que componen el escenario se puedan comunicar de forma segura con un solo nodo concreto. Para conseguir esto, cuando el controlador sabe todos los nodos que hay disponibles en el escenario, se comunica con ellos de forma individualizada, instanciando la configuración correspondiente para establecer el canal de comunicación seguro con el nodo que hará de central en la topología. Cuando este proceso finaliza, solo un nodo tiene canales seguros de comunicación con el resto, y por lo tanto, el resto solo tienen un canal seguro de comunicación, establecido con el nodo central. Como ejemplo, se puede ver un escenario con 5 nodos en la Figura 9.

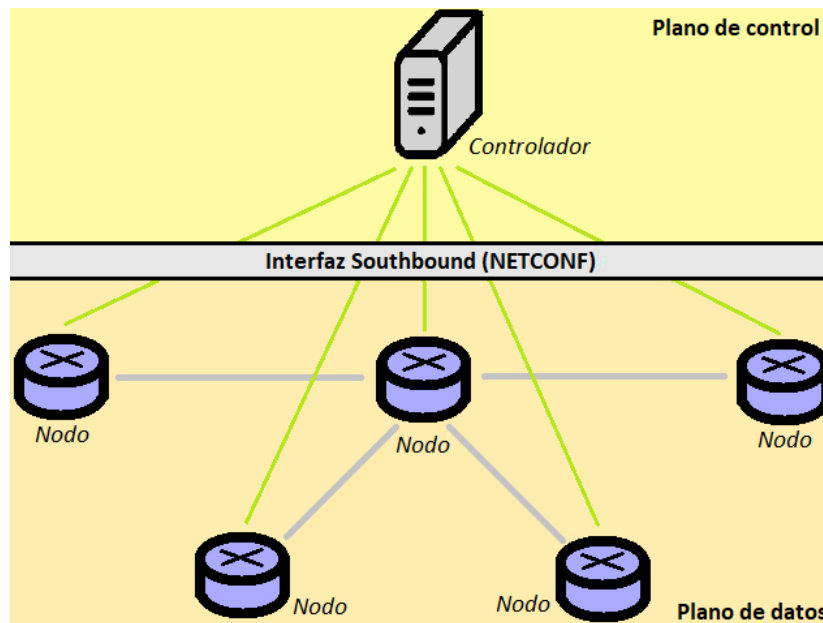


Figura 9: Representación de una topología estrella (con 5 nodos)

Partiendo de esta idea, se han realizado diferentes experimentos de manera similar al caso anterior, donde se pretende obtener resultados que reflejen el tiempo necesario para instanciar todas las configuraciones necesarias en los diferentes nodos de la red. Por lo tanto, queremos saber el tiempo que se ha de emplear para desplegar

una topología estrella securizada, donde se entiende este tiempo como la diferencia de tiempo entre que el controlador comienza la conexión con el primer nodo hasta que termina la conexión con el último.

Para obtener los tiempos deseados se han realizado una serie de experimentos siguiendo la misma dinámica que en el caso anterior. Se han realizado experimentos en escenarios que contienen desde 2 hasta 20 nodos, intentando representar con los tiempos obtenidos como se comporta el escenario en función del número de nodos. Se ha realizado 30 veces el experimento para cada número de nodos, obteniendo así valores más representativos que si se realizara la prueba solo una vez o un número inferior de veces. Los valores medios obtenidos para cada número de nodos se muestran en la Figura 10.

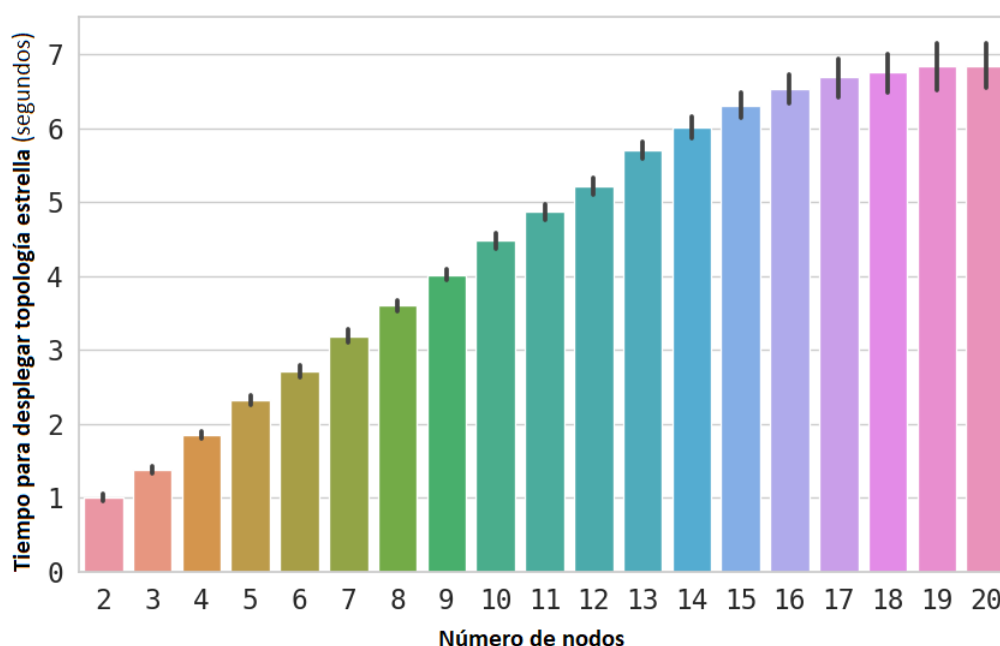


Figura 10: Tiempos medios para establecer una la topología estrella

Tras observar la gráfica de tiempos obtenida, se puede ver que tampoco sigue una distribución lineal. En este caso se observa una clara diferencia en la tendencia en comparación a los experimentos anteriores, ya que en los primeros se observaba una tendencia cuadrática, mientras que en este caso la tendencia presenta una distribución de forma logarítmica.

La diferencia entre estos dos casos radica en el aumento de asociaciones de seguridad a realizar conforme aumenta el número de nodos. En este caso, la configuración del escenario está diferenciada en dos partes, una primera parte de configuración del nodo que actúa de central frente a una segunda de configurar el resto de nodos. En

este escenario, cuanto más aumenta el número de nodos, más despreciable se hace la configuración del nodo central frente a los otros nodos ($n-1$ nodos). De esta manera se puede deducir que ahora el tiempo de configuración de $n-1$ sigue una distribución lineal. Para comprender mejor esto, es importante saber como aumenta el trabajo a realizar en función del número de nodos, es decir, como aumenta el número de políticas que se aplican y las conexiones asociadas a estas.

Partiendo de la implementación del controlador, cuando tiene constancia de todos los nodos que van a conformar la red y están preparados para recibir configuraciones, va procediendo con ellos de forma secuencial (tal y como se indicaba en el caso previo, realmente se lanzan en hilos paralelos pero esta parte se realiza en exclusión mutua, por lo que en la práctica se realiza de forma secuencial). En esta implementación, cuando se tiene que configurar el primer nodo, se considera este como el nodo central, de tal manera que el resto de nodos establecerán canales seguros de comunicación solo con este nodo central. Por lo tanto, cuando entra el segundo nodo, simplemente se debe establecer un canal seguro con el nodo central. Cuando entre un tercero, este no deberá establecer canales seguros con los dos nodos previos, sino solo con el nodo central, de la misma manera que lo ha hecho el anterior, y así sucesivamente.

En base a este comportamiento, se puede deducir que cada vez que entra un nodo nuevo a configurarse con el resto de nodos que ya están configurados, el número de canales de comunicación bidireccionales que se deben crear aumenta en uno, que sumado a los que ya había configurados, siempre tendremos un canal de comunicación menos que el número de nodos de la topología. Por lo tanto, siendo n el número de nodos, las comunicaciones bidireccionales que se deben crear son simplemente:

$$(n - 1)$$

A diferencia del caso *mesh* expuesto previamente, en este caso sí se puede ver que el número de conexiones tiene un aumento lineal, conforme aumenta el número de nodos (evidente una vez se comprende que se deben generar $n-1$ asociaciones de seguridad). Sin embargo, el aumento que poseen los tiempos de generar la topología estrella no siguen una distribución lineal, donde la distribución y la progresión de los tiempos obtenidos se puede ver en detalle en la Figura 11. En la gráfica se representan todos los tiempos obtenidos en todas las pruebas (30 pruebas por cada número de nodos), en forma de puntos, donde se observa una mayor dispersión que en el caso anterior.

El comportamiento que poseen los tiempos obtenidos, se debe a que en los tiempos influyen más características aparte del aumento de las configuraciones a aplicar. Como se comentaba también en el caso anterior, también influye la realización de todas las aplicaciones de políticas de forma paralela en distintos hilos de procesamiento. Otro tiempo que influye es el necesario para generar las conexiones con cada

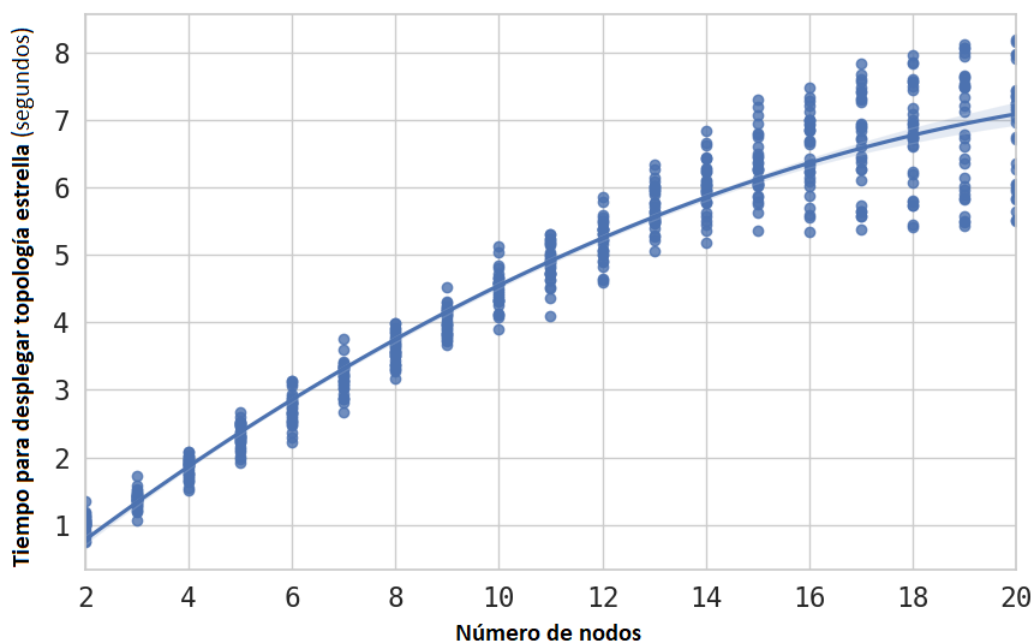


Figura 11: Tiempos obtenidos en todas las pruebas realizadas para obtener la topología estrella completa, junto a la progresión obtenida

nodo, sobre las que se instanciará la información necesaria en cada caso.

En estas pruebas se obtienen tiempos bastante inferiores que los obtenidos en el caso anterior, lo que lleva también a que se observe un rango de dispersión mayor en los resultados obtenidos. De la misma manera, al tener tiempos bastante más bajos, el tiempo que se invierte en establecer las conexiones juega un papel más relevante. Por lo tanto, cuando se realizan pruebas con un número bajo de nodos el tiempo total posee una mayor parte de establecimiento de conexiones. Finalmente, conforme se realizan las conexiones e instanciaciones de forma paralela, al no tener un aumento de los canales bidireccionales a crear (recordemos que ahora sí sigue una distribución lineal), los tiempos se irán reduciendo en función del aumento de los nodos (se entiende reduciendo como situarse por debajo de la distribución lineal esperada teniendo en cuenta solo el número de canales a crear).

Los tiempos que se han obtenido en la realización de este experimento son también bastante estables, donde el tiempo de formar una topología estrella es de aproximadamente unos 7 segundos. Por lo tanto, para obtener el tiempo necesario (de forma aproximada) para formar una topología estrella de n nodos, se puede obtener un valor medio por asociación de seguridad, ya que disponemos de $(n-1)$ asociaciones de seguridad en el escenario.

6. Conclusiones y vías futuras

Durante el desarrollo de este trabajo se ha centrado el esfuerzo en mejorar la gestión de la seguridad IPSec en los nodos finales de la red, especialmente en escenarios que siguen el paradigma SDN. Sobre estos escenarios, una entidad central gestiona y distribuye los recursos a los diferentes nodos finales, ofreciendo una vista global y abstracta de toda la red. La inclusión de esta entidad, llamada controlador software, supone una mejora en gestión, eficiencia y velocidad de la red entre otras, tanto para los usuarios finales de esta como para las empresas del mismo ámbito. Estas características, junto a la pobre optimización que posee SDN en aspectos relacionados con la seguridad, han sido la motivación para el desarrollo de este trabajo.

El punto de partida ha sido el trabajo previo realizado por el autor de este mismo trabajo. En él, se conseguía incluir un módulo en los nodos finales que aplicaba las configuraciones automáticamente en el mismo momento que las recibía y las procesaba. Sin embargo, la parte del controlador se realizaba manualmente, siendo en este punto donde se enfoca el desarrollo del trabajo actual.

Se ha analizado e incluido un nuevo módulo en el nodo controlador que permite instanciar en los nodos finales las configuraciones de forma automática. El módulo es capaz de saber los nodos que hay disponibles en el escenario, comunicarse con ellos e instanciar políticas de seguridad cuando sea necesario. De esta manera, ya no se requiere intervención externa en el escenario para tener securizadas las conexiones internas del escenario. También se ha analizado el modelo YANG utilizado para validar la estructura de los mensajes intercambiados durante las comunicaciones en el escenario. Por otra parte, también se ha adaptado el escenario para permitir realizar las pruebas con números de nodos variables, a diferencia del escenario estático con únicamente dos nodos del que se disponía al principio del desarrollo.

Una vez está disponible el escenario, se han realizado sobre él diferentes pruebas experimentales con la idea de obtener el rendimiento ofrecido por el escenario, en relación a los tiempos necesarios para establecer canales seguros de comunicación entre los nodos que conforman la red. Se han desplegado dos topologías diferentes basándose en el mismo escenario. Una primera que tiene como objetivo establecer canales seguros de comunicación entre todos los nodos que componen la topología frente a una segunda que pretende establecer los canales de comunicación seguros solo entre un nodo y el resto, teniendo un nodo que actúa como central.

Una vez se han obtenido y desarrollado los resultados experimentales, se puede observar como en la primera topología el tiempo se incrementa de una manera más abultada que el crecimiento del número de nodos de la topología, es decir, que conforme aumenta el número de nodos el tiempo necesario para establecer todos los canales de comunicación seguros aumenta de una manera más pronunciada (dis-

tribución cuadrática). Sin embargo, para la segunda topología sucede al contrario, donde al aumentar el número de nodos el tiempo para disponer de una topología securizada aumenta en menor medida (distribución logarítmica). Estos resultados se deben, principalmente, a que en la primera topología conforme se aumenta el número de nodos crece de una manera más abultada el número de canales a crear (recordamos, del orden $O(n^2)$ de la manera $n(n-1)/2$, siendo n el número de nodos), mientras que en la segunda crecen ambos de manera lineal (del orden de $n-1$), entrando en juego el paralelismo en la instanciación de las diferentes políticas.

Por último, en relación a las futuras vías de desarrollo, quedan abiertas todavía diferentes caminos a seguir para mejorar el trabajo y los resultados obtenidos. Por ejemplo, se podría investigar en relación al envío y recepción de notificaciones de los nodos cuando suceda cualquier evento, como la expiración de algunas asociaciones de seguridad, o también investigar la nueva instanciación de políticas cuando expiren las asociaciones de seguridad o las negociaciones de IKE establecidas. También sería una buena mejora el despliegue de diferentes topologías en el escenario aparte de las dos que se han desarrollado y con las que se ha experimentado en este trabajo (como un recorrido concreto dentro de una topología), u obtener otras métricas aparte de los tiempos expuestos (como tiempos necesarios para realizar una nueva negociación e instanciación cuando expire una clave).

Referencias

- [1] *Draft* relacionado con IPSec, en el que se explican en más detalle los ejemplos comentados. <https://datatracker.ietf.org/doc/html/draft-ietf-i2nsf-sdn-IPSec-flow-protection-08>
- [2] Trabajo Fin de Grado desarrollado por José Padilla Martínez. *Módulo en Python para gestionar asociaciones de seguridad IPSec usando SDN e IKE en dispositivos de red.*
- [3] *Draft* relacionado con la protección IPSec, creado por Rafael Marín López & Gabriel López Millán, ubicado en <https://tools.ietf.org/html/draft-ietf-i2nsf-sdn-IPSec-flow-protection-00>
- [4] Estándar SDN en IETF. <https://tools.ietf.org/html/rfc7426>
- [5] Estándar NETCONF en IETF. <https://tools.ietf.org/html/rfc6241>
- [6] Estándar RPC en IETF. <https://tools.ietf.org/html/rfc5531>
- [7] Estándar XML en IETF. <https://tools.ietf.org/html/rfc4826>
- [8] Estándar IPSec en IETF. <https://tools.ietf.org/html/rfc6071>
- [9] Estándar AH en IETF. <https://tools.ietf.org/html/rfc4302>
- [10] Estándar MD5 en IETF. <https://tools.ietf.org/html/rfc1321>
- [11] Estándar SHA1 en IETF. <https://tools.ietf.org/html/rfc3174>
- [12] Estándar ESP en IETF. <https://tools.ietf.org/html/rfc4303>
- [13] Estándar 3DES en IETF. <https://tools.ietf.org/html/rfc1851>
- [14] Estándar AES en IETF. <https://tools.ietf.org/html/rfc3826>
- [15] Estándar relacionado con asociaciones de seguridad y su arquitectura. <https://tools.ietf.org/html/rfc4301>
- [16] Imagen obtenida de <https://lira.epac.to/DOCS-TECH/Security/Internet%20Security%20Technology%20and%20Hacking/final/ch15lev1sec2.html>.
- [17] Estándar IKEv2 en IETF. <https://tools.ietf.org/html/rfc5996>
- [18] Estándar Diffie-Hellman en IETF. <https://tools.ietf.org/html/rfc2631>
- [19] Draft en proceso de estandarización desarrollado por Gabriel López Millán, Rafael Marín López y Fernando Pereñíguez García. <https://datatracker.ietf.org/doc/html/draft-ietf-i2nsf-sdn-IPSec-flow-protection-14>

-
- [20] Artículo desarrollado por Gabriel López Millán, Rafael Marín López y Fernando Pereñíguez García. <https://www.sciencedirect.com/science/article/abs/pii/S0920548918303052>
- [21] Trabajo Fin de Grado desarrollado por Ignacio Martínez Alpiste. *Configuración centralizada de IKE en múltiples dispositivos con el uso de SDNs*